

Asymmetric Multi-Model AI Routing Engine (aMMare)

A Containerized Agentic AI Toolchain for Local Execution, Context Compression, Model Routing, and Frontier Validation leveraging [VLLM](#), [LiteLLM](#), and [Headroom](#).

Pronunciation Guide

Ammare (or aMMare) is pronounced /'æm.ɑ:ɹ/

Syllable 1 (/ 'æm/): Say the word "am" (as in, "I am"). Make sure the "a" sound is flat and sharp, like the "a" in cat or trap. Drop your jaw slightly and push the sound from the front of your mouth.

Syllable 2 (/ .ɑ:ɹ/): Say the word "marr" (as in, "This is a non-marring mallet, as I do not want to mar the new flooring during installation?"). Pronounce the M, then make a wide, open throat sound like the "ah" sound you make at the doctor, smoothly blending into a standard English "R" sound (like at the end of car or far).

Table of Contents

[Pronunciation Guide](#)

[Table of Contents](#)

[Executive Summary](#)

[Global Traffic Architecture](#)

[Agent Execution Flow](#)

[Component Responsibilities](#)

[User Interface Layer](#)

[OpenHands Container](#)

[Containerized LangChain Agent Middleware Layer](#)

[Containerized Headroom Proxy Layer](#)

[Containerized LiteLLM Proxy Layer](#)

[Containerized Local Model Endpoint using vLLM](#)

[Frontier Model Cloud API](#)

[Responsibility Boundary Summary](#)

[Tool Execution and Safety Model](#)

[Core Execution Principle](#)

[Tool Execution Boundary](#)

[Approved Tool Registry](#)

[Default Deny Model](#)

[Human Approval Gates](#)

[Read-Only First Behavior](#)

[Command Execution Controls](#)

[File Access Controls](#)

[Secret Handling](#)

[Network Access Controls](#)

[Tool Result Filtering](#)

[Audit Logging](#)

[Failure Handling](#)

[Change Validation](#)

[Git and Change Control](#)

[Container Isolation](#)

[Privilege Separation](#)

[Policy Configuration](#)

[Safety Profile Modes](#)

[Final Response Requirements](#)

[Summary](#)

[Model Routing and Escalation Strategy](#)

[Routing Objective](#)

[Asymmetric Model Design](#)

[Primary Routing Layer](#)

[Example Model Tiers](#)

[Local-First Routing](#)

[Escalation Triggers](#)

[Confidence-Based Escalation](#)

[Tool-Aware Routing](#)

[State-Aware Routing](#)

[Task Classification](#)

[Risk-Based Routing](#)

[Cost and Latency Controls](#)

[Context Preparation Before Escalation](#)

[De-Escalation](#)

[Multi-Model Review](#)

[Fallback Behavior](#)

[Routing Decision Logging](#)

[Policy-Driven Routing](#)

[Example Routing Flow](#)

[Summary](#)

[Development and Deployment Roadmap](#)

[Roadmap Principles](#)

[Phase Zero: Repository Scaffold and Baseline Standards](#)

[Phase One: Local LLM Endpoint](#)

[Phase Two: LangChain Middleware Layer](#)

[Phase Three: Direct Local Model Workflow Validation](#)

[Phase Four: LiteLLM Routing Layer](#)

[Phase Five: Cloud Model Provider Integration](#)

[Phase Six: Routing and Escalation Logic](#)

[Phase Seven: Headroom Integration](#)

[Phase Eight: OpenHands Integration](#)

[Phase Nine: Memory, Context, and Retrieval](#)

[Phase Ten: Full Service Chain Validation](#)

[Phase Eleven: One-Click Modular Deployment](#)

[Phase Twelve: Documentation, Hardening, and Release Packaging](#)

[Summary](#)

[Target Service Topology](#)

[Topology Principles](#)

[Target Service Chain](#)

[Logical Endpoint Strategy](#)

[Proposed Service Inventory](#)

[Required Initial Services](#)

[Optional Services](#)

[Service Naming Standard](#)

[Port Management](#)

[Network Model](#)

[Shared Configuration Paths](#)

[Shared Runtime Paths](#)

[Environment Variable Standards](#)

[Startup Order](#)

[Independent Troubleshooting Paths](#)

[Health Check Standard](#)

[Logging Standard](#)

[Deployment Profiles](#)

[Topology Update Requirement](#)

[Summary](#)

[Component Configuration Model](#)

[Configuration Principles](#)

[Configuration Layout](#)

[Global Configuration](#)

[Component Configuration](#)

[Logical Endpoint Configuration](#)

[Model Endpoint Registry](#)

[Routing Configuration](#)

[Secrets Handling](#)

[Configuration Validation](#)

[Configuration Ownership](#)

[Summary](#)

[Service Chaining and Rewiring Strategy](#)

[Chaining Principle](#)

[Logical Gateway Pattern](#)

[Phase-Based Chain Evolution](#)

[Rewiring Events](#)

[Rewiring Requirements](#)

[Bypass Paths](#)

- [Deployment Profiles](#)
- [Avoiding Early Hardcoding](#)
- [Service Contract Expectations](#)
- [Configuration-Driven Chain Selection](#)
- [Validation After Rewiring](#)
- [Failure Isolation](#)
- [Documentation Requirement](#)
- [Summary](#)

[One-Click Deployment and Modular Install Strategy](#)

- [Deployment Principle](#)
- [Top-Level Deployment Script](#)
- [Component Deployment Scripts](#)
- [Deployment Profiles](#)
- [Deployment Order](#)
- [Preflight Checks](#)
- [Idempotency Requirements](#)
- [Optional Component Flags](#)
- [Validation During Deployment](#)
- [Uninstall and Reset Behavior](#)
- [Generated Files](#)
- [Deployment Summary Output](#)
- [Coding Agent Expectations](#)
- [Summary](#)

[Helper Scripts and Operational Tooling](#)

- [Tooling Principles](#)
- [Recommended Script Categories](#)
- [Core Validation Scripts](#)
- [Service-Chain Validation](#)
- [Endpoint Testing Scripts](#)
- [Model Management Scripts](#)
- [Routing Management Scripts](#)
- [Log Inspection Scripts](#)
- [Support Bundle Script](#)
- [Service Lifecycle Scripts](#)
- [Coding Agent Support](#)
- [Output Standards](#)
- [Documentation Requirement](#)
- [Summary](#)

[Documentation Strategy](#)

[Documentation Principles](#)

[Recommended Documentation Layout](#)

[Main README](#)

[Architecture Documentation](#)

[Component Documentation](#)

[Modification Guides](#)

[Deployment Documentation](#)

[Configuration Documentation](#)

[Validation and Troubleshooting Documentation](#)

[Coding Agent Documentation](#)

[Documentation Update Requirement](#)

[Avoiding Redundancy](#)

[Summary](#)

[Coding Agent Implementation Guide](#)

[Implementation Principles](#)

[Phase-Based Development](#)

[Required Work Pattern](#)

[Repository Awareness](#)

[Configuration Rules](#)

[Service Chain Rules](#)

[Script Standards](#)

[Deployment Script Expectations](#)

[Validation Requirements](#)

[Documentation Requirements](#)

[Error Handling Expectations](#)

[Security Expectations](#)

[Reporting Requirements](#)

[Stop Conditions](#)

[Summary](#)

[Memory, Context, and Retrieval Strategy](#)

[Strategy Principle](#)

[Session Context](#)

[Runtime Task State](#)

[Retrieved Context](#)

[Persistent Memory](#)

[Memory Storage Rules](#)

[Retrieval Backend](#)

- [Embedding Model](#)
- [Context Injection Policy](#)
- [Source Tracking](#)
- [Memory and Routing Interaction](#)
- [Memory and Tool Safety](#)
- [Indexing Process](#)
- [Retrieval Validation](#)
- [Reset and Purge Behavior](#)
- [Documentation Requirement](#)
- [Summary](#)

[Deployment Model](#)

- [Deployment Goals](#)
- [Initial Deployment Target](#)
- [Expanded Deployment Target](#)
- [Container Runtime](#)
- [Network Model](#)
- [IP Address and DNS Strategy](#)
- [Persistent Data](#)
- [Configuration and Secrets](#)
- [Deployment Profiles](#)
- [Host Assumptions](#)
- [Deployment Boundary](#)
- [Summary](#)

[Validation and Success Criteria](#)

- [Primary Success Criteria](#)
- [Validation Principles](#)
- [Phase-Level Success Criteria](#)
- [Configuration Validation](#)
- [Component Validation](#)
- [Service-Chain Validation](#)
- [Model Routing Validation](#)
- [Tool Execution Validation](#)
- [Memory and Retrieval Validation](#)
- [Deployment Validation](#)
- [Documentation Validation](#)
- [Final System Success Criteria](#)
- [Failure Criteria](#)
- [Summary](#)

[Deferred v2 Features](#)

[Deferral Principle](#)

[Deferred Features List](#)

[Not Deferred](#)

[Conditional Features](#)

[Re-Evaluation Criteria](#)

[Summary](#)

[Client Tool Integration Strategy](#)

[Integration Goal](#)

[Gateway Compatibility](#)

[Client Configuration Scripts](#)

[Manual Configuration Documentation](#)

[Example Configuration Values](#)

[Integration Profiles](#)

[Tool-Specific Notes](#)

[Validation for Client Integrations](#)

[Safety and Permissions](#)

[Documentation Requirement](#)

[Summary](#)

Executive Summary

The **Asymmetric Multi-Model AI Routing Engine, or aMMare**, is a containerized agentic AI toolchain designed to provide a practical, local-first development assistant that can use both local and cloud-hosted AI models through a controlled service chain.

The purpose of aMMare is not simply to run a local chatbot. A local chatbot can answer questions, explain code, and generate examples, but it does not automatically provide the full set of developer actions needed for useful automation. The goal of aMMare is to build a system that can assist with real development workflows, including reading and writing files, generating code changes, running scripts, reviewing command output, troubleshooting failures, running tests, inspecting logs, and optionally preparing or committing changes to Git.

This distinction is central to the architecture. A language model by itself does not directly modify files, run shell commands, monitor logs, or execute tests. The model generates text,

structured responses, code, diffs, or tool-call instructions. The actual execution of local actions is handled by the agent layer. In this design, that responsibility belongs primarily to the LangChain Agent Middleware Layer.

The aMMare service chain is designed around the following major components:

User Interface Layer — The interface where the user submits requests and reviews results. This may be a web UI, CLI, IDE integration, OpenHands, or another front-end interface, such as an IDE or a CLI tool

OpenHands — A containerized, browser-accessible agentic development environment. OpenHands can function as a web-based IDE or agent workspace, giving the user a visual interface for agent-driven development tasks. It may be used as one of the user-facing interfaces in the aMMare stack, especially for workflows where a sandboxed browser-based development environment is preferred.

LangChain Agent Middleware Layer — The agent orchestration layer. This is responsible for maintaining state, defining tools, validating model responses, executing approved local actions, capturing results, and deciding whether the task should continue, retry, fail, or escalate to another model. Without this layer, an LLM is just a chatbot. We have chosen to use LangChain as the orchestration layer rather than rely on internal IDE tooling which would tie the end user to a specific User Interface

Headroom Proxy Layer — A context optimization proxy. This layer is intended to reduce token usage and improve efficiency by compressing or optimizing payloads before they are sent to downstream model-routing infrastructure.

LiteLLM Proxy Layer — The model gateway and routing layer. LiteLLM (<https://www.litellm.ai/>) provides a unified OpenAI-compatible interface to local and frontier models. It handles model abstraction, routing, fallback behavior, virtual keys, usage tracking, budget controls, and provider management.

Local Model Endpoint — A locally hosted model served through vLLM. This provides private, zero-marginal-cost inference for routine coding tasks, drafting, code explanation, simple edits, and other workloads that can be handled locally.

Frontier Model Cloud API — One or more cloud-hosted frontier models used for complex reasoning, final validation, architectural review, difficult debugging, or fallback when the local model is not sufficient.

The aMMare architecture is based on a physical service-chain model. Requests flow from the user-facing layer into **LangChain**, then through **Headroom**, then through **LiteLLM**, and finally to either a local **vLLM-hosted model** or a **frontier cloud model**. Responses return through the same proxy chain back to **LangChain**, where they are validated and acted upon.

This means that **LangChain** is not just passing messages back and forth. It is the control point for the agentic workflow. If the model returns a normal answer, LangChain may return that answer to the user. If the model returns a valid tool-call instruction, LangChain may execute the appropriate tool. If the model returns an invalid or unsafe response, LangChain can reject it, retry the request, route to a different model, or return an error to the user.

The architecture solves several problems at once.

- First, it separates model inference from tool execution. This avoids the incorrect assumption that a local model must somehow have built-in access to the filesystem or shell. Instead, the local model only needs to generate useful responses or structured tool calls, while LangChain handles the actual actions.
- Second, it provides a local-first model strategy. Routine work can be handled by a local vLLM-hosted model, reducing dependency on paid APIs and preserving privacy for appropriate tasks.
- Third, it preserves access to frontier models when needed. Complex reasoning, architectural validation, security review, and failed local loops can be escalated to a stronger cloud model through LiteLLM.
- Fourth, it introduces governance around model usage. LiteLLM provides a central control plane for model access, virtual keys, spend tracking, quota management, and fallback routing. This is important because agentic workflows can generate repeated model calls, and uncontrolled loops could quickly consume paid API quota.
- Fifth, it introduces a path for token optimization. Headroom can be placed inline as a proxy to reduce context size before requests reach LiteLLM and downstream models. This may help lower cloud model costs and may also make local model usage more practical when context size is a limiting factor.
- Sixth, it creates a flexible interface strategy. aMMare does not need to depend on a single user interface. OpenHands can provide a containerized browser-based agent workspace. A custom web UI or CLI can provide a simpler interface. IDE integrations may be added later. The key architectural point is that these interfaces connect into the same controlled service chain.

The initial POC of aMMare should focus on proving the core service chain with one local model, one LangChain Agent Middleware Layer, Headroom as an inline proxy, LiteLLM as the routing gateway, and at least one frontier model for fallback or validation. More advanced features, such as multiple local model pipelines, complex LangGraph workflows, long-term

memory, or automated multi-agent review loops, should be treated as later enhancements after the core system is stable.

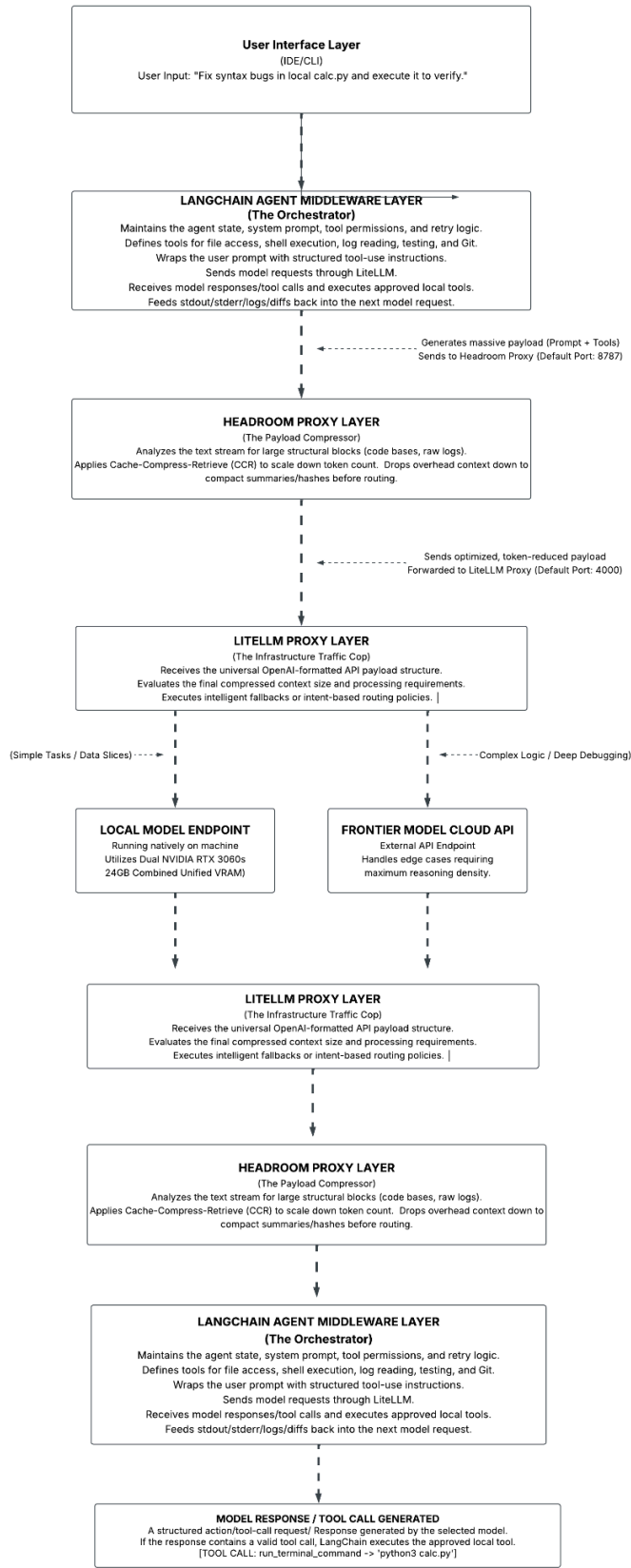
In short, aMMare is being built to turn local and cloud language models into a controlled, agentic development toolchain. The system is intended to provide the practical functionality users expect from modern AI development assistants, while preserving local execution, cost control, model flexibility, and clear separation of responsibilities between the user interface, agent layer, proxy layers, and model endpoints.

Global Traffic Architecture

The Global Traffic Architecture diagram represents the physical service chain for aMMare. Each box in the diagram represents a deployed service, container, or external API dependency. The diagram is not intended to show every internal process step inside the agent loop. Internal workflow details, such as tool validation, file writes, shell execution, retry behavior, and observation handling, are described in later sections.

See Diagram Below

LOCAL AI AGENT ARCHITECTURE & PROXY CHAIN FLOW DIAGRAM



Agent Execution Flow

The **Global Traffic Architecture section** describes the physical service chain. This section describes what happens inside that service chain when a user submits a request that requires agentic behavior.

The most important distinction is that the service chain and the agent execution flow are related, but they are not the same thing. The service chain describes which deployed services the request passes through. The agent execution flow describes how the LangChain Agent Middleware Layer interprets the request, communicates with the model, validates the model response, performs approved actions, and decides what to do next.

In aMMare, the LangChain Agent Middleware Layer is the component responsible for turning a user request into an actionable workflow.

A user may submit a request such as:

“Fix syntax bugs in calc.py and execute it to verify.”

This request cannot be completed by a language model alone. The model can inspect code, suggest changes, generate a patch, or request that a command be run, but the model does not directly access the local filesystem or shell. The LangChain Agent Middleware Layer provides that missing execution capability.

The high-level execution flow is:

1. The user submits a request through the **User Interface Layer**.
2. The request is sent to the **LangChain Agent Middleware Layer**.
3. **LangChain** builds the agent context. This may include the user request, system instructions, available tools, current task state, workspace path, prior observations, and any relevant file content.
4. **LangChain** sends the model request through the service chain.
5. The request passes through **Headroom** for context optimization, then through **LiteLLM** for model routing.
6. **LiteLLM** routes the request to either the local vLLM model endpoint or a frontier model cloud API.
7. The selected model generates a response.
8. The response returns through **LiteLLM**, then through **Headroom**, and back to **LangChain**.
9. **LangChain** validates the model response.

10. If the model response is a normal answer, **LangChain** may return it to the User Interface Layer.
11. If the model response contains a valid tool call, **LangChain** executes the approved local tool.
12. If the model response is invalid, unsafe, incomplete, or not usable, **LangChain** may retry, request correction, escalate to a frontier model, or stop the workflow and report failure.
13. If a tool is executed, **LangChain** captures the result as an observation. This may include stdout, stderr, exit code, file diff, test result, log output, or Git status.
14. **LangChain** adds the observation to the current agent state.
15. **LangChain** decides whether the task is complete or whether another model call is required.
16. The loop continues until the task succeeds, fails safely, reaches a retry limit, or requires user input.

The agent loop is what gives aMMare its practical value. Without this loop, the system would only generate code examples or recommendations. With this loop, aMMare can make controlled local changes, run commands, inspect failures, and iterate.

The **LangChain Agent Middleware Layer** must therefore be designed as a stateful execution controller, not as a simple pass-through API. It needs to understand the task objective, maintain context, track tool usage, validate model responses, enforce limits, and determine when to stop.

For example, if a model returns a tool call requesting that `calc.py` be executed, LangChain should not blindly execute it without validation. LangChain should verify that the requested tool exists, that the command is allowed, that the working directory is permitted, and that the action fits within the current task policy. Once approved, LangChain executes the command and captures the result.

If the command fails with a syntax error, LangChain does not treat the task as complete. Instead, it sends the error output back through the model chain with an updated prompt explaining what failed. The model can then generate a correction, and LangChain can apply the correction and rerun the command.

This process continues until the task is complete or until LangChain determines that it should stop.

The agent execution flow should also support escalation. If the local model fails repeatedly, produces invalid tool calls, or cannot resolve the issue, LangChain should be able to request assistance from a frontier model through LiteLLM. This allows aMMare to remain local-first without becoming local-only.

The intended behavior is not that every task automatically goes to the strongest model. Instead, LangChain should use the local model where practical and reserve frontier models for cases where additional reasoning, validation, or recovery is needed.

In summary, the Agent Execution Flow defines how aMMare turns model output into controlled action. The model provides reasoning and instructions. LangChain validates those instructions, executes approved tools, captures results, and continues the workflow until the task reaches a safe and useful conclusion.

Component Responsibilities

This section defines the responsibility of each containerized component in the aMMare service chain. Each component has a specific role. The intent is to avoid mixing user interface behavior, agent orchestration, context optimization, model routing, and model inference into a single unclear layer.

The core service chain is composed of the following major components:

- User Interface Layer
- OpenHands Container
- Containerized LangChain Agent Middleware Layer
- Containerized Headroom Proxy Layer
- Containerized LiteLLM Proxy Layer
- Containerized Local Model Endpoint using vLLM
- Frontier Model Cloud APIs

The architecture should be understood as a group of cooperating services. The User Interface Layer receives requests and displays results. OpenHands provides a browser-based agent workspace. The LangChain Agent Middleware Layer owns the custom aMMare agent workflow and executes approved tools. Headroom optimizes context. LiteLLM routes and governs model traffic. vLLM provides local inference. Frontier model APIs provide fallback, validation, and high-complexity reasoning.

User Interface Layer

The User Interface Layer is the entry point where the user submits requests and reviews results.

This layer may be implemented through one or more interfaces, including a web UI, CLI, IDE integration, OpenHands, or a future custom aMMare interface. The architecture should not depend on only one user interface. Multiple interfaces should be able to connect into the same backend service chain.

The User Interface Layer is responsible for accepting user input, displaying task status, showing final results, and presenting relevant outputs such as summaries, diffs, command output, errors, or logs.

The User Interface Layer does not need to directly execute tools. In the custom aMMare workflow, tool execution is handled by the Containerized LangChain Agent Middleware Layer. This keeps the interface simple and prevents the UI from becoming responsible for filesystem access, shell execution, or retry logic.

Example user request:

“Fix the syntax error in custom.py and run it to verify the fix.”

The User Interface Layer passes this request into the service chain. It does not need to know whether the request will be handled by the local model, a frontier model, or a retry sequence involving both.

OpenHands Container

OpenHands is included as a containerized, browser-accessible agentic development environment. It can function as a web-based IDE or agent workspace where the user can interact with an autonomous coding agent through a visual interface.

Within aMMare, OpenHands should be treated as one possible user-facing interface and agent workspace. It can connect to the same model infrastructure through LiteLLM, allowing OpenHands to use local models, frontier models, or routed model configurations.

OpenHands may have its own internal agent behavior. This means it should be documented separately from the custom LangChain Agent Middleware Layer. In workflows where OpenHands is used directly, OpenHands may own more of the agent loop. In workflows using the custom aMMare UI or CLI, the Containerized LangChain Agent Middleware Layer owns the agent loop.

For the first implementation, OpenHands should be deployed as a separate container and integrated into the model service chain. It should not replace the LangChain middleware path. Both can exist in the stack as different ways to interact with the model and agent infrastructure.

Containerized LangChain Agent Middleware Layer

The Containerized LangChain Agent Middleware Layer is the primary orchestration component for the custom aMMare workflow.

This service is responsible for turning a user request into a controlled agentic workflow. It maintains task state, prepares prompts, defines available tools, validates model responses, executes approved local actions, captures execution results, and determines the next step in the workflow.

This layer owns tool execution for the custom aMMare agent path. That means it may perform approved actions such as reading files, writing files, running shell commands, running tests, inspecting logs, checking Git status, generating diffs, or preparing commits.

The language model does not directly execute these actions. The model only generates a response, patch, diff, or tool-call instruction. The LangChain service receives that model output, validates it, and decides whether the requested action is allowed.

For example, if the model returns a request to run:

python3 calc.py

The LangChain Agent Middleware Layer validates the request before executing it. The service should confirm that the requested command is permitted, that it is being run in an approved workspace, and that it fits within the current task policy. If approved, LangChain executes the command from the configured workspace and captures stdout, stderr, the exit code, and any relevant output.

That execution result becomes an observation. LangChain can then return the result to the user, send it back to the model for additional troubleshooting, retry the task, escalate to a frontier model, or stop the workflow if the task has failed safely.

This container should include the custom aMMare agent logic, tool definitions, execution policy, retry limits, prompt templates, workspace configuration, and integration code for sending model requests through Headroom and LiteLLM.

The LangChain container should also define the boundaries of the local workspace. It should know which directories it is allowed to read from, which directories it is allowed to write to, and which commands are permitted. This is important because the model may request actions that are malformed, unnecessary, unsafe, or outside the intended task scope.

The LangChain Agent Middleware Layer should not blindly execute anything the model returns. It must validate model responses and reject malformed, unsafe, or out-of-policy tool calls. If a

response is invalid, the LangChain service can retry, ask the model to correct its format, escalate to a frontier model, or stop and return an error to the user.

This layer is also responsible for managing the agent loop. A single user request may require multiple model calls and multiple local actions. For example, LangChain may ask the model to inspect a file, apply a change, run a command, capture an error, send that error back to the model, apply another change, and rerun the command. The workflow continues until the task is complete, the retry limit is reached, or the system determines that user input is required.

In summary, the Containerized LangChain Agent Middleware Layer is the control point that turns model output into safe, structured, and auditable action. It is the component that makes the system agentic. The model provides reasoning and instructions, but LangChain validates those instructions, performs approved actions, captures observations, and manages the workflow through completion

Containerized Headroom Proxy Layer

The Containerized Headroom Proxy Layer is an inline context optimization service positioned between the Containerized LangChain Agent Middleware Layer and the Containerized LiteLLM Proxy Layer.

Its purpose is to reduce token usage and optimize request payloads before they reach the model-routing layer. This may help lower frontier model cost and may also improve local model usability when context size becomes a limiting factor.

The expected request path is:

```
LangChain Agent Middleware Layer => Headroom Proxy Layer => LiteLLM Proxy Layer.
```

The expected response path is:

```
LiteLLM Proxy Layer => Headroom Proxy Layer => LangChain Agent Middleware Layer.
```

Headroom should be treated as a proxy service, not as the agent orchestrator. It does not own tool execution, local file changes, retry policy, task state, model selection, or provider routing decisions. Those responsibilities remain with LangChain and LiteLLM.

Headroom's primary responsibility is context optimization. In practical terms, this may include compressing large prompts, reducing repeated context, shortening long conversation history, or optimizing the amount of information sent downstream to the selected model.

This layer is potentially useful because agentic workflows can generate large context payloads. Repeated command output, file content, error traces, prior model responses, and task history can quickly increase token usage. If every model call includes the full uncompressed context, the system may become slower, more expensive, or less reliable.

However, Headroom must be tested carefully because agentic workflows are sensitive to exact formatting. It should not corrupt or rewrite content that must remain exact, including tool-call schemas, JSON structures, file paths, shell commands, code blocks, patches, diffs, stdout, stderr, traceback output, or test results.

For this reason, Headroom should support a bypass or disable option for workflows where exact formatting is more important than token reduction. For example, if a workflow depends on structured tool calls or precise command output, LangChain may need to send the request through the service chain without compression.

For the initial version of aMMare, Headroom should be deployed as a containerized proxy, but its behavior should be validated before it is treated as mandatory for all workflows. The system should be designed so that Headroom can be enabled, disabled, or bypassed without requiring major changes to the rest of the architecture.

In summary, the Containerized Headroom Proxy Layer provides context optimization between LangChain and LiteLLM. It is intended to improve efficiency and reduce token usage, but it must preserve the exact information required for reliable agent execution.

Containerized LiteLLM Proxy Layer

The Containerized LiteLLM Proxy Layer is the model gateway for aMMare.

Its purpose is to give the rest of the system one consistent endpoint for model access. Instead of configuring LangChain, OpenHands, and other tools to talk directly to different models, those tools send their model requests to LiteLLM.

LiteLLM then decides which configured model backend should receive the request. That backend may be the local vLLM model, a frontier cloud model, or another configured provider.

LiteLLM is responsible for model routing, fallback behavior, virtual keys, budget controls, rate limits, and usage tracking.

LiteLLM does not execute local actions. It does not write files, run shell commands, inspect logs, modify code, or manage the agent loop. Those responsibilities belong to the LangChain Agent Middleware Layer, or to OpenHands when OpenHands is being used directly.

In the aMMare service chain, LiteLLM receives requests from Headroom and routes them to the selected model. When the model responds, the response returns to LiteLLM first, then flows back through Headroom to LangChain.

The initial LiteLLM configuration should include at least two model routes: one local model route pointing to vLLM, and one frontier model route pointing to a cloud provider. Additional routes can be added later.

LiteLLM should also issue separate virtual keys for different clients, such as LangChain and OpenHands. This allows aMMare to track usage by client and apply different permissions, budgets, or rate limits.

In short, LiteLLM is the control point for model access. It lets aMMare use local and cloud models through one managed interface while providing routing, fallback, cost control, and visibility.

Containerized Local Model Endpoint using vLLM

The Containerized Local Model Endpoint using vLLM provides self-hosted model inference for aMMare.

This service hosts the selected local model and exposes an OpenAI-compatible API endpoint that LiteLLM can route to. The purpose of this layer is to provide local model capacity for routine development tasks without requiring every request to use a paid frontier model.

The local vLLM endpoint may be used for tasks such as code drafting, code explanation, simple bug fixes, README updates, shell command suggestions, boilerplate generation, and other development work that does not require advanced frontier-model reasoning.

The local model should not be treated as the component that performs local actions. It does not directly write files, run commands, inspect logs, or modify a repository. It only generates model output. That output may be a normal answer, a code suggestion, a patch, a diff, or a

tool-call instruction. The LangChain Agent Middleware Layer is responsible for validating and acting on that output.

For the first version of aMMare, the local model strategy should use one local model endpoint. This keeps the design simpler and allows the project to focus on proving the service chain, agent loop, and tool execution model before adding more local model complexity.

The local model should be evaluated based on how well it works inside the actual aMMare workflow, not only on benchmark scores. Important criteria include stable vLLM startup, acceptable latency, usable context length, ability to produce valid structured responses, compatibility with LangChain tool-call patterns, and ability to reason over command output or errors returned by the agent layer.

The local model endpoint should be considered replaceable. If the first selected model is not reliable enough for the desired workflow, the architecture should allow a different model to be deployed behind vLLM without requiring major changes to LangChain, Headroom, LiteLLM, or the user interface.

In summary, the Containerized Local Model Endpoint using vLLM provides the local inference capability for aMMare. It supports the local-first strategy, but it does not replace the agent layer. The model generates responses and instructions; LangChain performs approved local actions.

Frontier Model Cloud API

The Frontier Model Cloud API represents **one or more** external high-capability model providers.

Unlike the other major components, this is not deployed as a local aMMare container. It is an external API dependency accessed through LiteLLM.

Frontier models are used for tasks where the local model is not sufficient. These may include complex debugging, architecture review, security analysis, final code review, difficult failure recovery, and fallback when the local model produces invalid or low-quality responses.

The purpose of frontier models is not to replace the local-first strategy. Their purpose is to provide controlled escalation. LiteLLM should manage access to these models through configured routes, virtual keys, rate limits, budget controls, and usage tracking.

This allows aMMare to use local inference for routine work while preserving access to stronger models when needed.

Responsibility Boundary Summary

- The User Interface Layer accepts user requests and displays results.
- The OpenHands Container provides a browser-based agent workspace and may act as a user-facing development environment.
- The Containerized LangChain Agent Middleware Layer owns the custom aMMare agent loop, tool execution, response validation, state management, retries, and observations.
- The Containerized Headroom Proxy Layer optimizes context and reduces token usage as an inline proxy.
- The Containerized LiteLLM Proxy Layer routes model traffic, manages providers, applies fallback, tracks usage, and enforces budget controls.
- The Containerized Local Model Endpoint using vLLM provides local model inference.
- The Frontier Model Cloud API provides high-capability reasoning, validation, and fallback through an external provider.

This separation of responsibilities is essential to the aMMare design. It ensures that each service has a clear purpose and prevents confusion between interface, orchestration, context optimization, routing, inference, and tool execution.

Tool Execution and Safety Model

The aMMare platform separates model reasoning from tool execution.

The language model is allowed to reason, plan, generate responses, propose file changes, request tool usage, and produce structured instructions. It is not allowed to directly execute commands, modify files, access credentials, alter runtime state, or interact with infrastructure on its own.

All tool execution is mediated by the Containerized LangChain Agent Middleware Layer. This layer acts as the controlled execution boundary between the model and the local system.

The purpose of this design is to allow agentic workflows while preserving operational control, auditability, repeatability, and safety.

Core Execution Principle

The model may suggest an action, but the middleware decides whether that action is allowed, how it is executed, and what result is returned.

This creates a clear separation of responsibility:

- The model performs reasoning and produces intent.
- The middleware validates and constrains that intent.
- Approved tools perform the actual action.
- Results are captured and returned to the model as bounded context.

This prevents the model from becoming an unrestricted shell, file editor, or automation engine.

Tool Execution Boundary

The tool execution boundary is enforced inside the Containerized LangChain Agent Middleware Layer.

This boundary controls access to approved capabilities such as:

- Reading project files.
- Writing or modifying approved files.
- Running limited shell commands.
- Running tests or validation scripts.
- Inspecting logs.
- Checking Git status.
- Generating diffs.
- Preparing commits.
- Querying approved local services.
- Calling approved APIs.
- Interacting with approved model endpoints.

The model does not execute these operations directly. Instead, it emits a structured request, and the middleware determines whether the request is valid.

For example, the model may request to inspect a repository file, run a test command, or apply a patch. The middleware evaluates that request against the configured policy before performing the action.

Approved Tool Registry

The middleware maintains an approved tool registry.

The registry defines which tools are available to the agentic workflow and what each tool is permitted to do. Each tool should have a narrow, clearly defined purpose.

Examples of registered tools may include:

- File read tool.
- File write tool.
- Patch application tool.
- Shell command tool.
- Test runner tool.
- Git status tool.
- Git diff tool.
- Git commit preparation tool.
- Log inspection tool.
- Configuration validation tool.
- Local API request tool.
- Model endpoint request tool.

Each tool should define:

- Its allowed input format.
- Its allowed execution scope.
- Its permitted working directories.
- Whether it can modify state.
- Whether it requires human approval.
- What output is returned to the model.
- What output is written to logs.
- What failure conditions must stop the workflow.

This registry prevents tool access from being implied or open-ended.

Default Deny Model

The tool execution model should follow a default deny approach.

If a requested action is not explicitly allowed, it is rejected.

The middleware should not assume that a command, path, API call, or file operation is safe simply because the model requested it. Safe behavior must be defined in policy.

Examples of rejected actions may include:

- Commands outside the approved command allowlist.
- File writes outside the approved project workspace.
- Attempts to read secrets, tokens, SSH keys, or credential files.
- Attempts to access host-level system paths.
- Attempts to modify container runtime configuration without approval.
- Attempts to execute network calls to unapproved destinations.
- Attempts to install packages without approval.
- Attempts to delete files without approval.
- Attempts to rewrite Git history.
- Attempts to modify production infrastructure directly.

This approach allows the system to support useful automation without granting unrestricted local authority.

Human Approval Gates

Some actions should require explicit human approval before execution.

Human approval gates should be used for any action that is destructive, security-sensitive, infrastructure-impacting, externally visible, or difficult to reverse.

Examples of actions that should require approval include:

- Deleting files.
- Overwriting existing configuration files.
- Installing packages.
- Changing firewall rules.
- Modifying container runtime settings.
- Restarting services.
- Applying infrastructure changes.
- Sending outbound messages or API requests.
- Creating Git commits.
- Pushing to a remote repository.
- Running commands with elevated privileges.
- Accessing sensitive logs or restricted files.

The middleware should pause the workflow and present the proposed action in a clear, reviewable format. The user can then approve, reject, or modify the action.

Approval should be action-specific. Approval for one action should not grant broad approval for future unrelated actions.

Read-Only First Behavior

Agentic workflows should begin in read-only mode whenever possible.

The middleware should first allow the model to inspect files, gather context, understand project structure, review logs, and evaluate current state. Write actions should occur only after the model has enough context to justify the change.

This helps prevent premature edits and reduces the risk of the model making incorrect assumptions about the environment.

A typical safe workflow should follow this pattern:

- Inspect relevant files and current state.
- Summarize findings.
- Propose a plan.
- Request approval for state-changing actions.
- Apply approved changes.
- Validate the result.
- Report what changed and what remains.

This pattern is especially important when working with deployment scripts, container definitions, CI workflows, security settings, and infrastructure automation.

Command Execution Controls

Shell command execution should be tightly constrained.

The shell command tool should not provide unrestricted terminal access. Instead, it should enforce command-level policy before execution.

Controls may include:

- Command allowlists.
- Argument validation.
- Working directory restrictions.

- Timeout limits.
- Output size limits.
- Environment variable filtering.
- Network access restrictions.
- Privilege restrictions.
- Denial of shell chaining where appropriate.
- Denial of interactive commands.
- Denial of commands that require unrestricted TTY access.

The middleware should treat shell access as one of the highest-risk tools in the platform.

Commands such as file listing, Git status, test execution, linting, and project-local validation may be allowed with low friction. Commands that modify system state, install software, alter permissions, or affect services should require explicit approval.

File Access Controls

File access should be scoped to the approved workspace.

The middleware should define which directories the agent may read from and write to. In most cases, this should be limited to the mounted project directory and selected temporary working paths.

The model should not be able to freely read from the host filesystem.

Restricted paths should include:

- SSH keys.
- API tokens.
- Shell history.
- Browser profiles.
- Password stores.
- System credential files.
- Host runtime sockets.
- Private keys.
- Cloud provider credential files.
- Unrelated user directories.
- Sensitive operating system paths.

Write access should be more restrictive than read access. A file may be safe to inspect but unsafe to modify.

The middleware should also preserve file change visibility by generating diffs before and after modifications.

Secret Handling

The model should not be given direct access to secrets.

Secrets may be required by the middleware or tools to interact with approved services, but those secrets should not be exposed back into the model context unless explicitly sanitized.

Secret handling should follow these principles:

- Do not place raw secrets in prompts.
- Do not return raw secrets in tool output.
- Redact known credential patterns from logs.
- Restrict access to secret files.
- Use environment variables or mounted secrets only where required.
- Avoid storing secrets in generated artifacts.
- Block attempts to print, summarize, transform, or exfiltrate secrets.

If a tool requires a credential, the tool should use that credential internally and return only the result required for the workflow.

Network Access Controls

Network access should be explicit and policy-controlled.

The middleware should define which network destinations the agentic workflow may access. This may include approved internal services, local model endpoints, documentation sources, source control systems, or specific APIs.

Unrestricted outbound network access should be avoided.

Network policy should account for:

- Approved destinations.
- Approved ports.
- Approved protocols.
- Whether authentication is required.
- Whether requests are read-only or state-changing.
- Whether external calls require approval.
- Whether the response can be returned to the model.

This is important because network access can become an exfiltration path, a command-and-control path, or an unintended production-impact path.

Tool Result Filtering

Tool output should be filtered before it is returned to the model.

The model only needs the information required to continue the task. It does not need unrestricted access to full command output, full logs, full environment dumps, or complete file trees.

Filtering may include:

- Truncating excessive output.
- Removing secrets.
- Removing irrelevant environment variables.
- Summarizing long logs.
- Returning only matching lines from large files.
- Redacting tokens, keys, and passwords.
- Blocking binary output.
- Separating user-visible output from internal diagnostic output.

This reduces context noise and helps prevent sensitive information from being pulled into the model session.

Audit Logging

All tool activity should be logged.

Audit logs should record what the model requested, what the middleware allowed or rejected, what tool was executed, what inputs were used, what outputs were returned, and whether human approval was required.

Audit logging should include:

- Request timestamp.
- Session identifier.
- Tool name.
- Requested action.
- Approved or rejected status.
- Approval source, if applicable.

- Working directory.
- Command or operation summary.
- Exit code or result status.
- Error message, if applicable.
- Redacted output summary.
- File paths affected.
- Diff summary for file modifications.

Audit logs should not store raw secrets.

The audit trail is important for troubleshooting, security review, reproducibility, and understanding how a generated change was produced.

Failure Handling

Tool execution failures should be handled explicitly.

The middleware should capture failures and return structured results to the model. The model can then reason about the failure and propose a correction, but it should not be allowed to repeatedly execute risky actions without control.

Failures may include:

- Invalid tool input.
- Policy rejection.
- Command timeout.
- Non-zero command exit.
- Permission failure.
- Missing file.
- Failed validation.
- Failed test run.
- API error.
- Network timeout.
- Unsafe output detection.

For repeated failures, the middleware should be able to stop the workflow and ask for human direction.

This prevents runaway loops where the model repeatedly retries commands or makes increasingly broad changes to resolve an issue.

Change Validation

State-changing actions should be followed by validation.

After modifying files, applying a patch, updating configuration, or preparing a deployment change, the middleware should run approved validation steps where available.

Validation may include:

- Syntax checks.
- Unit tests.
- Linting.
- ShellCheck.
- YAML validation.
- JSON validation.
- Markdown validation.
- Container Quadlet validation.
- Dry-run deployment commands.
- Git diff review.
- Security scanning.
- Project-specific health checks.

The validation result should be returned to the model and included in the final task summary.

If validation fails, the model may propose a corrective change, but the correction should follow the same tool execution and approval rules as the original change.

Git and Change Control

Git operations should be handled carefully.

The middleware may allow the model to inspect Git status, review diffs, and prepare commit messages. However, committing, tagging, pushing, rebasing, resetting, or altering history should require explicit user approval. If a user requests git commit, or git push actions, the user should be prompted to allow such actions.

A safe Git workflow should include:

- Inspect current branch.
- Inspect working tree state.
- Review changed files.
- Generate a diff.

- Summarize intended changes.
- Run validation.
- Run pre-commit
- Prepare a commit message.
- Request approval before committing.
- Request approval again before pushing.

This ensures that the user remains in control of repository history and remote publication.

Container Isolation

The middleware and tool execution services should run inside containers.

Container isolation provides an additional boundary between the agentic workflow and the host system. The container should receive only the mounts, environment variables, network access, and permissions required for the task.

The container should avoid unnecessary privileges, and be configured to start up in the proper order upon system reboot.

Recommended controls include:

- Non-root container users where possible.
- Read-only mounts where practical.
- Narrow bind mounts.
- No host root filesystem mount.
- No unrestricted container socket access.
- Limited Linux capabilities.
- SELinux enforcement where applicable.
- Resource limits for CPU and memory.
- Network segmentation.
- Explicit volume definitions.
- Controlled access to model endpoints and local APIs.

Container isolation is not a complete security boundary by itself, but it reduces the blast radius of tool execution.

Privilege Separation

The platform should separate privileged operations from normal agentic operations.

Most workflows should run with non-privileged permissions. Privileged actions should be routed through narrowly scoped tools or approval gates.

For example, a normal file inspection tool should not have the same authority as a service restart tool. A Git diff tool should not have the same authority as a Git push tool. A test runner should not have the same authority as a package installation tool.

This separation allows the platform to grant the minimum authority required for each action.

Policy Configuration

Tool policy should be configurable rather than hardcoded into model prompts.

The model prompt may describe expected behavior, but enforcement must happen in middleware policy.

Policy configuration may define:

- Allowed tools.
- Allowed paths.
- Allowed commands.
- Denied commands.
- Approved network destinations.
- Human approval requirements.
- Maximum output size.
- Maximum runtime.
- File write restrictions.
- Secret redaction rules.
- Logging requirements.
- Retry limits.
- Environment-specific restrictions.

This allows different deployments of aMMare to use different safety profiles.

For example, a personal homelab deployment may allow broader local file access inside a project directory, while a shared enterprise deployment may require stricter approval gates, tighter network policy, and more detailed audit logging.

Safety Profile Modes

The platform may support multiple safety profile modes.

Example modes include:

- Read-only mode.
- Local development mode.
- Assisted edit mode.
- Approved execution mode.
- Restricted infrastructure mode.
- Enterprise controlled mode.

Each mode can expose a different set of tools and approval requirements.

Read-only mode may allow file inspection and summarization only. Local development mode may allow tests and project-local edits. Restricted infrastructure mode may require approval for any state-changing action. Enterprise controlled mode may integrate with identity, policy, ticketing, and audit systems.

These modes make the platform adaptable without changing the core architecture.

Final Response Requirements

At the end of a workflow, the middleware should require the model to produce a clear final summary.

The final response should identify:

- What was inspected.
- What was changed.
- What was not changed.
- What validation was performed.
- Whether validation passed or failed.
- What risks or limitations remain.
- Whether human follow-up is required.
- Where relevant diffs, logs, or artifacts can be reviewed.

This ensures that the user receives an operationally useful result rather than a vague statement that the task is complete.

Summary

The aMMare tool execution and safety model is based on controlled delegation.

The model provides reasoning and intent, but the middleware owns execution control. Tools are explicitly registered, scoped, validated, logged, and constrained. Human approval is required for risky or irreversible actions. Secrets, file access, shell execution, network calls, and Git operations are all treated as controlled capabilities rather than unrestricted model permissions.

This design allows aMMare to support practical agentic workflows while maintaining clear boundaries around safety, accountability, and operational control.

Model Routing and Escalation Strategy

The aMMare platform uses an asymmetric model-routing strategy.

Not every request should be sent to the same model. Different tasks require different levels of reasoning, speed, cost, context length, tool access, and reliability. The routing layer decides which model should handle a request based on the type of work being performed and the level of confidence required.

This allows the platform to use smaller or local models for routine work while escalating complex, ambiguous, or high-risk work to stronger models when needed.

Routing Objective

The goal of model routing is to match each task to the most appropriate model tier.

The router should consider:

- Task complexity.
- Required reasoning depth.
- Required context length.
- Need for tool execution.
- Need for code understanding.
- Need for structured output.
- Sensitivity of the data.
- Latency requirements.
- Cost constraints.
- Confidence level of the first response.
- Whether the task affects system state.

This prevents the platform from overusing expensive or remote models while also avoiding weak responses for tasks that require stronger reasoning.

Asymmetric Model Design

The aMMare architecture assumes that different models have different roles.

A smaller local model may be well suited for:

- Simple classification.
- Intent detection.
- Request summarization.
- Basic question answering.
- Lightweight code explanation.
- Log summarization.
- Routing decisions.
- Drafting simple responses.
- Extracting structured fields.
- Running repetitive support tasks.

A larger or remote model may be better suited for:

- Complex reasoning.
- Architecture planning.
- Multi-step troubleshooting.
- Code generation.
- Security analysis.
- Ambiguous requests.
- Cross-file repository understanding.
- Explaining failures.
- Generating implementation plans.
- Reviewing diffs.
- Producing final user-facing summaries.

This approach allows the platform to use each model where it is strongest.

Primary Routing Layer

The primary routing layer receives the user request and determines the initial model path.

The router may be implemented as part of the LangChain Agent Middleware Layer or as a separate service in front of the model endpoints. In either case, the routing function should be policy-controlled and observable.

The router should evaluate the request before execution begins.

Example routing inputs include:

- User prompt.
- Session context.
- Available tools.
- Requested operation.
- Workspace metadata.
- Data sensitivity.
- Current safety profile.
- Prior model response quality.
- Tool results from earlier workflow steps.

The router should return a structured routing decision rather than relying on informal prompt behavior alone.

Example Model Tiers

The platform may define model tiers rather than binding the architecture to specific model names.

Example tiers include:

- Local lightweight model.
- Local coding model.
- Local reasoning model.
- Remote general-purpose model.
- Remote high-reasoning model.
- Specialized embedding model.
- Specialized reranking model.
- Specialized vision model.
- Specialized speech or transcription model.

The actual model assigned to each tier can change over time as models improve or deployment constraints change.

This keeps the architecture flexible and avoids hard-coding aMMare to one model vendor, one local runtime, or one API provider.

Local-First Routing

The default routing strategy should prefer local models when they are sufficient for the task.

Local-first routing provides several advantages:

- Lower operating cost.
- Lower latency for small tasks (ideally, target system spec dependent)
- Reduced dependency on external services.
- Better control over local data.
- Better offline or degraded-mode behavior.
- More predictable infrastructure behavior in a lab environment.

However, local-first does not mean local-only.

If the local model is unlikely to complete the task safely or accurately, the workflow should escalate to a stronger model tier.

Escalation Triggers

Escalation should occur when the platform detects that the current model tier is insufficient for the task.

Escalation triggers may include:

- Low model confidence.
- Repeated failed tool calls.
- Failed validation.
- Ambiguous user intent.
- Complex multi-step reasoning.
- Cross-file code analysis.
- Security-sensitive changes.
- Infrastructure-impacting changes.
- User-visible production changes.
- Requests involving unfamiliar frameworks.
- Large context requirements.
- Need for stronger code review.
- Need for policy interpretation.

- Need for final answer verification.

Escalation may also be requested by the model itself, but the middleware should decide whether escalation is allowed.

The model should not be trusted as the sole authority on whether it needs escalation.

Confidence-Based Escalation

The router should support confidence-based escalation.

A model response may be considered low confidence when it contains:

- Uncertainty about the requested task.
- Incomplete reasoning.
- Contradictory claims.
- Unsupported assumptions.
- Invalid tool requests.
- Repeated corrections.
- Failed tests.
- Output that does not match the requested schema.
- A result that conflicts with observed tool output.

When confidence is low, the middleware may reroute the task to a stronger model or request human clarification.

Confidence should be based on observable signals where possible, not only on the model's self-reported confidence.

Tool-Aware Routing

Routing should account for the tools required by the task.

Some models may be allowed to use only read-only tools. Other models may be allowed to propose write actions but not execute them. Stronger or more trusted model tiers may be allowed to participate in workflows that involve code edits, patch review, or infrastructure planning.

The tool execution policy remains enforced by the middleware regardless of which model is selected.

Model routing should not bypass tool safety policy.

A stronger model may produce a better plan, but it should still operate through the same approval gates, file restrictions, network restrictions, and audit logging controls.

State-Aware Routing

The router should consider the current workflow state.

For example, the initial request may be simple enough for a local lightweight model, but later tool output may reveal that the task is more complex. At that point, the middleware can escalate.

Examples include:

- A simple error message expands into a multi-service troubleshooting workflow.
- A file edit requires understanding multiple related configuration files.
- A failed test reveals a deeper logic problem.
- A deployment script change affects container runtime behavior.
- A log summary reveals security-sensitive findings.
- A documentation update becomes an architecture design task.

Routing should therefore be available throughout the workflow, not only at the first prompt.

Task Classification

The router should classify tasks before selecting a model.

Possible task classes include:

- General question.
- Documentation generation.
- Code explanation.
- Code generation/modification.
- Test generation.
- Test execution.
- Script execution and output analysis
- Log analysis.
- Configuration review.
- Infrastructure planning.
- Security review.
- Repository maintenance.
- Troubleshooting.

- Data extraction.
- Summarization.
- Final response generation.

Each task class can map to a preferred model tier, tool profile, and escalation rule.

For example, simple summarization may use a local lightweight model. Repository-wide code modification may start with a local coding model but require escalation to a stronger reasoning model if validation fails.

Risk-Based Routing

The routing strategy should account for operational risk.

Higher-risk tasks should be routed to stronger models and stricter safety profiles.

Examples of higher-risk tasks include:

- Security configuration changes.
- Authentication or authorization changes.
- Firewall changes.
- Container runtime changes.
- CI/CD pipeline changes.
- Infrastructure automation changes.
- Data deletion.
- Git history modification.
- External API calls.
- Production or shared-environment changes.

For high-risk tasks, the platform may require:

- Stronger model tier.
- Human approval.
- Read-only inspection before modification.
- Validation before final response.
- Diff review.
- Audit logging.
- Explicit rollback notes.

This ensures that routing decisions account for consequences, not only prompt complexity.

Cost and Latency Controls

Routing should balance answer quality with cost and latency.

The platform should avoid sending every request to the largest available model. Large models should be reserved for cases where their added reasoning quality is likely to matter.

Cost and latency controls may include:

- Local-first defaults.
- Token budget limits.
- Maximum context size per model tier.
- Escalation only after observable need.
- Use of summarization before escalation.
- Use of embeddings and retrieval to reduce context size.
- Caching for repeated queries.
- Model selection based on expected output size.
- Timeouts for slow model responses.

The purpose is not to minimize cost at all times. The purpose is to spend cost where it improves correctness, safety, or user value.

Context Preparation Before Escalation

Before escalating to a stronger model, the middleware should prepare a compact context package.

The stronger model should receive the information needed to continue the task without receiving unnecessary raw data.

The context package may include:

- Original user request.
- Current task classification.
- Current workflow state.
- Relevant file excerpts.
- Tool results.
- Error messages.
- Failed validation output.
- Current plan.
- Prior model response.
- Safety constraints.

- Required output format.
- Open questions or unresolved risks.

This keeps escalation efficient and reduces the chance of leaking irrelevant or sensitive information.

De-Escalation

The router should also support de-escalation.

After a complex issue has been resolved or a plan has been produced by a stronger model, routine follow-up work may return to a smaller local model.

Examples of de-escalation include:

- Formatting a final summary.
- Updating simple documentation.
- Running a known validation command.
- Extracting fields from tool output.
- Summarizing a completed diff.
- Producing a short user-facing status update.

De-escalation helps preserve resources while keeping high-reasoning models focused on the parts of the workflow where they are most useful.

Multi-Model Review

For certain workflows, aMMare may use more than one model to review the same output.

Multi-model review may be useful for:

- Security-sensitive changes.
- Large code modifications.
- Architecture documents.
- Complex troubleshooting plans.
- Generated deployment scripts.
- Policy-sensitive recommendations.
- Final review before commit or push.

One model may generate a plan or patch, while another model reviews it for correctness, safety, completeness, and policy alignment.

The middleware should treat multi-model review as an advisory process. Final execution authority remains with the middleware and the user approval process.

Fallback Behavior

The routing layer should define fallback behavior when a model is unavailable.

Fallback conditions may include:

- Local model endpoint unavailable.
- Remote API unavailable.
- Model timeout.
- Rate limit.
- Authentication failure.
- Context window exceeded.
- Unsupported input type.
- Tool compatibility issue.

Fallback options may include:

- Retry the same model.
- Route to another local model.
- Route to a remote model.
- Reduce context size and retry.
- Switch to read-only mode.
- Stop and ask for human direction.
- Return a partial result with a clear limitation.

Fallback behavior should be predictable and logged.

Routing Decision Logging

Every routing decision should be logged.

Routing logs should capture:

- Timestamp.
- Session identifier.
- Task classification.
- Selected model tier.
- Selected model endpoint.

- Reason for selection.
- Escalation trigger, if applicable.
- De-escalation trigger, if applicable.
- Safety profile.
- Tool profile.
- Token estimate.
- Latency result.
- Failure or fallback behavior.

These logs help tune the router over time and explain why a specific model was used for a specific task.

Policy-Driven Routing

Routing should be controlled by policy, not only by prompt instructions.

A routing policy may define:

- Which models are available.
- Which model tiers are allowed for each task class.
- Which data types may be sent to remote models.
- Which models may participate in tool workflows.
- Which tasks require escalation.
- Which tasks require human approval.
- Which tasks must remain local.
- Which fallback paths are allowed.
- Which logging fields are required.

This makes routing behavior consistent, auditable, and adjustable without rewriting the whole application.

Example Routing Flow

A typical routing flow may work as follows:

- The user submits a request.
- The middleware classifies the task.
- The router selects an initial model tier.
- The selected model produces a response, plan, or tool request.
- The middleware validates any requested tool action.
- Tool results are returned to the model.

- The middleware evaluates confidence, validation results, and risk.
- If needed, the task escalates to a stronger model.
- If the task becomes routine, it de-escalates to a smaller model.
- The final response is generated.
- Routing and tool activity are logged.

This flow allows aMMare to adapt during the task instead of making a single static model choice at the beginning.

Summary

The aMMare model-routing strategy is based on asymmetric use of model capabilities.

Smaller and local models handle routine, low-risk, and repetitive work. Stronger models are reserved for complex reasoning, high-risk changes, large context requirements, code review, security review, and final verification. Routing decisions are based on task class, risk, confidence, tool needs, context size, cost, latency, and policy.

Escalation and de-escalation allow the system to adapt as the workflow develops. The middleware remains responsible for policy enforcement, tool safety, logging, and final execution control regardless of which model is selected.

Development and Deployment Roadmap

The aMMare platform should be developed and deployed in controlled phases.

The initial implementation should start with the smallest useful service chain, validate that chain, and then add additional components only after the previous layer is working. This phased approach reduces troubleshooting complexity and prevents the project from becoming difficult to debug during early development.

The roadmap should be treated as both a development plan and a deployment plan. Each phase should produce a working system state, even if that state is not the final architecture.

The coding agent should not attempt to build the entire platform in one pass.

Roadmap Principles

The development process should follow these principles:

- Build one layer at a time.
- Validate each layer before adding the next one.
- Avoid hardcoding service relationships that will change later.
- Use stable logical endpoint names from the beginning.
- Keep each component independently deployable where practical.
- Keep deployment scripts idempotent.
- Keep configuration externalized.
- Document every component as it is added.
- Add troubleshooting scripts alongside each service.
- Prefer simple working behavior before advanced routing behavior.
- Avoid introducing memory, retrieval, or multi-agent behavior until the core model path is stable.

The goal is not only to reach the final architecture. The goal is to create a system that remains understandable and maintainable while it grows.

Phase Zero: Repository Scaffold and Baseline Standards

The first phase establishes the project structure, standards, and baseline automation.

This phase should not focus on model behavior yet. It should focus on making sure the repository has a clean structure that future phases can build on.

Required outcomes:

- Create the base repository structure.
- Create top-level **README.md**.
- Create **docs/** directory.
- Create **scripts/** directory.
- Create **config/** directory.
- Create **env/** directory for environment examples.
- Create **containers/ directory** for service definitions.
- Create **tests/** directory.
- Add baseline linting and formatting standards (pre-commit, rules.md)
- Add shell script style expectations.
- Add example environment files.
- Add basic preflight validation script.
- Add basic project health script.
- Add placeholder documentation for future components.
- Add .gitignore and ensure all *.env files, which configure IPs or contain secrets, are ignored by git; only the *.env.example templates are committed

- Any <component>.env file created should have a corresponding <component>.env.example template that is included in the repo and is updated with any and all parameters that the end user will need to include in the <component>.env file
- All .env files and templates should be well documented with inline comments

The coding agent should avoid implementing advanced logic in this phase, however the coding agent should be aware of the entire architecture and implementation plan. The purpose is to prepare the project so later changes have a consistent location and naming model.

Expected validation:

- Repository structure exists.
- Basic scripts run without syntax errors.
- Example environment files are present.
- Documentation placeholders are present.
- No secrets are committed.
- Git status is clean after generated files are reviewed.

Phase One: Local LLM Endpoint

This phase deploys the first local model endpoint. The agent can use the following repo as reference - <https://github.com/christopherpaquin/vllm-local-developer-stack>

The local model endpoint provides the first usable model backend for the platform. This may be implemented with **vLLM** or another local inference runtime selected for the initial build. Initial POC environment has 2x GPUs so multi-GPU tensor parallelism is required.

At this stage, the model endpoint does not need advanced routing. It only needs to expose a predictable API that can be validated by simple test requests.

Required outcomes:

- Deploy the local LLM container.
- Select Appropriate model for the target environment (POC will have 2x NVIDIA RTX 3060 GPUs with 24 GB of VRAM total across both GPUs)
- Define the local model service name.
- Define the local model listening port.
- Define model storage paths.
- Define GPU visibility and runtime requirements.
- Define local model environment variables.
- Create a local model deployment script.
- Create a local model validation script.

- Create basic request/response smoke test.
- Document how to change the local model.
- Document how to view logs.
- Document common startup failures.
- Ensure that the LLM container starts at boot
- Ensure that all documentation related to the operation, endpoints, unit tests, and helper scripts are developed.

The endpoint should be exposed through a configurable logical model gateway variable.

This value may be a **local IP address**, an **internal DNS name**, a **container service name**, or a **reverse-proxy** name depending on the deployment model. For the initial local lab deployment, using a static local IP address is acceptable.

An important design requirement is that LangChain must not hardcode a direct dependency on a specific model backend. LangChain should read the model gateway endpoint from configuration so the backend can later change from direct local model access to LiteLLM, Headroom or another routing layer without requiring application code changes.

Example logical target using a container or internal DNS name:

```
AMMARE_MODEL_GATEWAY_URL=http://ammare-litellm:4000
```

Example logical target using a local lab IP address:

```
AMMARE_MODEL_GATEWAY_URL=http://10.1.10.17:4000
```

In early phases of the POC, this endpoint may point directly to the local LLM service. In later phases, the same variable can be repointed to LiteLLM, Headroom, or another model gateway component.

This allows the service chain to evolve from:

```
LangChain → Local LLM
```

to:

```
LangChain → LiteLLM → Local LLM / Cloud LLM
```

and later to:

```
LangChain → Headroom → LiteLLM → Local LLM / Cloud LLM
```

Without requiring LangChain logic to be rewritten each time the backend service chain changes.

Expected validation:

- Local model container starts.
- GPU is visible to the container if required.
- Model loads successfully.
- Healthy endpoint responds.
- Basic inference request succeeds.
- Logs show no fatal startup errors.
- Validation script returns success.

Phase Two: LangChain Middleware Layer

This phase introduces the **LangChain** middleware service.

The LangChain layer becomes the main orchestration point for user requests, prompt construction, routing decisions, tool-use requests, and workflow state. In early phases, its behavior should remain simple.

The first version should prove that LangChain can receive a request, call the local model endpoint, and return a response.

Required outcomes:

- Deploy LangChain middleware container.
- Define LangChain service name.
- Define LangChain listening port.
- Configure LangChain to use the logical model gateway URL.
- Add a simple request API.

- Add basic prompt template handling.
- Add basic structured logging.
- Add a health endpoint.
- Add validation script.
- Add documentation for modifying LangChain prompts and rules.
- Add documentation for troubleshooting LangChain-to-model communication.

The LangChain service should not be tightly coupled to a specific local model endpoint. It should rely on configuration.

Expected validation:

- LangChain container starts.
- LangChain health endpoint responds.
- LangChain can reach the configured model endpoint.
- Simple prompt request succeeds.
- Failed model calls return clear errors.
- Logs identify the model endpoint being used.
- Validation script confirms LangChain-to-model path.

Phase Three: Direct Local Model Workflow Validation

This phase validates the simplest useful service chain.

The chain at this stage is:

```
User/API Client → LangChain Middleware → Local LLM Endpoint
```

The purpose of this phase is to prove the core request path before introducing additional routing or model gateway complexity.

Required outcomes:

- End-to-end request path works.
- LangChain can send prompts to the local model.
- Responses return to the user or API client.
- Basic error handling works.
- Service logs are useful for troubleshooting.
- Endpoint validation script confirms the full chain.
- Documentation describes the current direct service chain.

- Known limitations are documented.

Expected validation:

- End-to-end smoke test passes.
- Invalid request returns controlled error.
- Stopped model endpoint produces clear LangChain error.
- Restarted model endpoint recovers without manual reconfiguration.
- Logs show request path clearly.
- No cloud model or external dependency is required.

This phase establishes the minimum viable aMMare service path.

Phase Four: LiteLLM Routing Layer

This phase introduces LiteLLM as the model routing layer.

LiteLLM should become the model gateway between LangChain and one or more model providers. Initially, it may route only to the local model endpoint. The purpose is to introduce the routing abstraction before adding additional model providers.

The service chain becomes:

```
User/API Client → LangChain Middleware → LiteLLM → Local LLM Endpoint
```

Required outcomes:

- Deploy LiteLLM container.
- Define LiteLLM service name.
- Define LiteLLM listening port.
- Configure LiteLLM with local model backend.
- Repoint LangChain from direct local model access to LiteLLM.
- Preserve the same logical model gateway variable where possible.
- Add LiteLLM health check.
- Add LiteLLM routing validation script.
- Add documentation for adding and modifying LiteLLM model backends.
- Add troubleshooting documentation for LiteLLM-to-model failures.

This phase is important because it prevents LangChain from becoming responsible for every model provider connection directly.

Expected validation:

- LiteLLM container starts.
- LiteLLM can reach the local model backend.
- LangChain can reach LiteLLM.
- End-to-end request succeeds through LiteLLM.
- LiteLLM logs show backend routing activity.
- Direct local model access can be tested independently.
- Service chain validation confirms the new path.

Phase Five: Cloud Model Provider Integration

This phase adds the first cloud model provider behind LiteLLM.

The goal is to prove that the system can route to both a local model and a cloud model without changing LangChain's core request logic.

Required outcomes:

- Add cloud model provider configuration.
- Store cloud provider settings in external configuration.
- Store credentials securely outside committed files.
- Add model alias for the cloud model.
- Add validation script for cloud provider connectivity.
- Add documentation for adding additional cloud providers.
- Add documentation for required LangChain changes, if any.
- Add policy notes for what data may be sent to cloud models.
- Add failure behavior for unavailable cloud providers.

Expected validation:

- LiteLLM can reach the local model backend.
- LiteLLM can reach frontier cloud model backend.
- A request can be routed to the local model.
- A request can be routed to the frontier cloud model.
- Missing or invalid cloud credentials produce clear errors.
- LangChain does not require major code changes to use the new backend.
- Documentation explains how to add another provider.

! WARNING: Cloud credentials/APIs are secrets and should never be committed to git and should only exist locally on the target system

This phase should not yet attempt advanced automated escalation. It should first prove that multiple backends are reachable and selectable.

Phase Six: Routing and Escalation Logic

This phase implements the model routing and escalation strategy.

The system should begin making controlled routing decisions based on task type, risk level, confidence, context size, and configured policy. Routing should remain observable and explainable, and well documented.

Required outcomes:

- Define routing policy file.
- Define model tiers or aliases.
- Add task classification logic.
- Add local-first routing behavior.
- Add escalation triggers.
- Add fallback behavior.
- Add routing decision logs.
- Add routing validation tests.
- Add documentation for modifying routing rules.
- Add examples of simple, medium, and escalated requests.

The routing logic should remain configuration-driven where possible. The coding agent should avoid burying routing rules deep inside application code if they are likely to change. Whenever possible, helper-scripts should provide the end user a standardized way to add/modify/delete/validate routing logic. Helper scripts should exist in a service specific script directory under /scripts in the base repo

Expected validation:

- Simple requests route to the local model.
- Configured complex requests route to the stronger model.
- Failed local model requests can fallback or escalate.
- Routing decisions are logged.
- Routing policy can be modified without rewriting core code.

- Validation scripts show which model handled the request.

Phase Seven: Headroom Integration

This phase introduces Headroom into the service chain.

Headroom should be added only after LangChain, LiteLLM, local model routing, and cloud model routing are working. Adding Headroom earlier would make troubleshooting more difficult because failures could occur across too many layers at once.

The intended service chain may become:

```
User/API Client → LangChain Middleware → Headroom → LiteLLM → Local LLM  
/ Cloud LLM
```

The exact placement of Headroom should be confirmed during implementation. The architecture should preserve enough flexibility to place Headroom where it provides the intended routing, orchestration, or gateway behavior without forcing major rewrites.

Required outcomes:

- Define Headroom role in the service chain.
- Deploy Headroom container.
- Define Headroom service name.
- Define Headroom listening port.
- Configure upstream and downstream endpoints.
- Rewire LangChain or gateway configuration as needed.
- Preserve logical endpoint abstraction.
- Add Headroom health check.
- Add service chain validation through Headroom.
- Add troubleshooting documentation.
- Document how to bypass Headroom for isolation testing.

Expected validation:

- Headroom container starts.
- Headroom health endpoint responds.
- Headroom can reach LiteLLM.
- LangChain can reach Headroom if Headroom is in the request path.
- End-to-end request succeeds through the full chain.

- The previous LiteLLM path can still be validated independently.
- Logs identify where requests are passing or failing.

This phase should be treated as a controlled service-chain rewiring event.

Phase Eight: OpenHands Integration

This phase introduces OpenHands or a similar coding-agent workspace component.

OpenHands should not be introduced until the core model routing path is stable. It adds a new level of agentic behavior and may require workspace mounts, tool execution boundaries, and additional security controls.

Required outcomes:

- Define OpenHands role in the architecture.
- Define whether OpenHands is user-facing, middleware-facing, or task-specific.
- Deploy OpenHands container.
- Define workspace mount model.
- Define access boundaries.
- Define allowed repositories or project paths.
- Define relationship to LangChain middleware.
- Define relationship to model routing layer.
- Add OpenHands health check.
- Add basic coding-agent workflow validation.
- Add documentation for safe workspace usage.

Expected validation:

- OpenHands container starts.
- Workspace mounts are correct.
- OpenHands can reach the intended model endpoint or middleware layer.
- OpenHands cannot access unintended host paths.
- Basic coding-agent workflow succeeds.
- Logs and artifacts are visible for review.
- Tool execution restrictions are documented.

This phase should remain conservative. OpenHands should not receive broad host access simply because it is useful for coding tasks.

Phase Nine: Memory, Context, and Retrieval

This phase introduces persistent memory, document retrieval, or context indexing.

Memory and retrieval should be added after the core routing path is stable. Adding memory too early can make it harder to distinguish model behavior problems from retrieval quality problems.

Required outcomes:

- Define memory scope.
- Define what should and should not be stored.
- Define vector database or retrieval backend if used.
- Define document ingestion path.
- Define embedding model.
- Define retrieval policy.
- Define context injection rules.
- Define redaction behavior.
- Define reset or purge process.
- Add memory validation script.
- Add retrieval quality tests.
- Add documentation for adding documents to retrieval.
- Add documentation for troubleshooting bad retrieval results.

Expected validation:

- Memory service starts if applicable.
- Documents can be indexed.
- Retrieval returns relevant context.
- Irrelevant context is not over-injected.
- Sensitive content handling is documented.
- LangChain can use retrieved context.
- The system still works when retrieval is disabled.

Memory should be optional during initial deployment.

Phase Ten: Full Service Chain Validation

This phase validates the complete intended service path.

The final service chain may include:

```
User/API Client
→ LangChain Middleware
→ Headroom
→ LiteLLM
→ Local LLM and Cloud LLM providers
→ Optional memory/retrieval services
→ Optional coding-agent workspace
```

The exact final chain may vary based on which components are enabled.

Required outcomes:

- Add full service-chain validation script.
- Add component-by-component validation.
- Add dependency validation.
- Add configuration validation.
- Add model backend validation.
- Add routing validation.
- Add memory/retrieval validation if enabled.
- Add OpenHands validation if enabled.
- Add failure-mode tests.
- Add final deployment summary output.

Expected validation:

- Every enabled container is running.
- Every enabled health endpoint responds.
- Internal service DNS or hostnames resolve.
- Config files are valid.
- Required ports are listening.
- Required volumes exist.
- Model backends respond.
- End-to-end prompt succeeds.
- Routing behavior matches policy.
- Optional components can be disabled cleanly.
- Logs identify the active service path.

This phase should make it easy to determine whether the system is healthy without manually inspecting every container.

Phase Eleven: One-Click Modular Deployment

This phase consolidates earlier deploy scripts into a top-level one-click deployment process.

The one-click deployment script should orchestrate the component deployment scripts. It should not become a large monolithic script containing all logic.

Required outcomes:

- Create top-level deploy script.
- Create top-level uninstall script.
- Create top-level validation script.
- Add component selection flags.
- Add deployment profiles.
- Add preflight checks.
- Add ordered deployment logic.
- Add post-deployment validation.
- Add deployment summary.
- Add failure reporting.
- Add documentation for deployment profiles.

Example deployment profiles may include:

```
minimal-local  
local-with-litellm  
hybrid-local-cloud  
full-agentic  
full-agentic-with-memory
```

The deployment configuration should allow users to pick which components to deploy either directly on the cli, or via variable in the global environment file (env/ammare.env)

Expected validation:

- Minimal deployment works.
- Full deployment works.
- Optional components can be skipped.
- Re-running deployment is safe (idempotent)
- Failed deployment reports the failed component.
- Validation can be run independently after deployment.

- Uninstall behavior is documented and controlled.
- The creation of an uninstall script is imperative in order to allow for easy cleanup after a deployment (failed or successful). As we progress through the deployment phases, the uninstall script should allow the environment to be wiped clean in order to test deployment in a new environment minus any remaining deployment artifacts.

The top-level script should preserve modularity.

Phase Twelve: Documentation, Hardening, and Release Packaging

This phase prepares the project for repeatable use.

By this point, the system should already work. This phase focuses on making it understandable, maintainable, and safer for future modification.

Required outcomes:

- Complete top-level README.md.
- Complete component documentation.
- Complete deployment documentation.
- Complete configuration documentation.
- Complete troubleshooting documentation.
- Complete validation documentation.
- Complete coding-agent implementation guide.
- Review security-sensitive defaults.
- Review secret handling.
- Review container permissions.
- Review exposed ports.
- Review logging behavior.
- Review failure behavior.
- Add release checklist.
- Add known limitations.
- Add deferred features list.

Expected validation:

- A new user can understand the architecture from the README.
- A coding agent can follow the implementation guide.

- Each component has modification instructions.
- Each component has validation instructions.
- Troubleshooting scripts are documented.
- No secrets are committed.
- Default deployment is conservative.
- Final service-chain validation passes.

The aMMare development and deployment roadmap is intentionally phased.

The project should begin with a simple local model endpoint and LangChain middleware path. After that path is validated, LiteLLM can be added as the routing layer, followed by cloud model providers, escalation logic, Headroom, OpenHands, memory, and full-chain validation.

This phased approach provides a clear path to the final architecture while reducing early troubleshooting complexity. It also gives both the end user and the coding agent a practical roadmap for building, validating, documenting, and operating the system incrementally.

Detailed implementation expectations for the coding agent, including work patterns, configuration rules, validation requirements, and stop conditions, are defined in the Coding Agent Implementation Guide chapter later in this document.

Target Service Topology

The aMMare platform should define the intended service topology before all components are implemented.

The initial deployment may start with only a local model endpoint and LangChain middleware, but the architecture should still describe the final expected service layout. This gives the coding agent a clear target and prevents early implementation choices from making later service-chain changes more difficult.

The topology described in this section should be treated as the intended end state. Individual deployment phases may enable only a subset of these services.

Topology Principles

The service topology should follow these principles:

- Each major function should run as a separate containerized service.

- Each service should have a predictable name.
- Each service should have clearly documented listening ports.
- Internal service relationships should be configured through environment variables or config files.
- Direct hardcoded service URLs should be avoided inside application code.
- Optional services should be disabled cleanly when not deployed.
- Each service should have a health check.
- Each service should write logs to a predictable location.
- Each service should have its own documentation.
- The full service chain should be testable end to end.
- Each layer should be independently testable for troubleshooting.
- Each service should have non-ephemeral storage (when storage is required)
- Each service should start automatically on boot
- NOTE: Additional consideration regarding startup order may be required depending on service behavior or dependencies

The topology should support both a minimal local deployment and a more complete multi-service deployment.

Target Service Chain

The final intended service chain may include the following path:

```
User/API Client
→ LangChain Middleware
→ Headroom
→ LiteLLM
→ Local LLM and Cloud LLM Providers
```

Optional supporting services may include:

```
Memory / Retrieval Backend
Embedding Model
Vector Database
OpenHands / Coding Agent Workspace
Management and Validation Scripts
```

The exact request path may vary depending on which deployment profile is enabled.

A minimal deployment may use:

```
User/API Client  
→ LangChain Middleware  
→ Local LLM Endpoint
```

A local routing deployment may use:

```
User/API Client  
→ LangChain Middleware  
→ LiteLLM  
→ Local LLM Endpoint
```

A hybrid deployment may use:

```
User/API Client  
→ LangChain Middleware  
→ LiteLLM  
→ Local LLM Endpoint / Cloud Model Provider
```

A full deployment may use:

```
User/API Client  
→ LangChain Middleware  
→ Headroom  
→ LiteLLM  
→ Local LLM Endpoint / Cloud Model Provider
```

The topology should allow these modes without requiring major application rewrites.

Logical Endpoint Strategy

Services should communicate through configurable logical endpoints.

A logical endpoint is a configuration value that points to the current target service. It may use a static local IP address, an internal DNS name, a container service name, or a reverse-proxy address.

For the local lab deployment, static local IP addresses are acceptable. The important requirement is that service targets are defined in configuration rather than hardcoded into application logic. Some services/containers will need LAN local IP addresses so that external users or tools may connect over the network leverage the stack, however many components will only need system local ip addresses and may not need an IP/port exposed to the LAN

Example using a local lab IP address:

```
AMMARE_MODEL_GATEWAY_URL=http://10.1.10.17:4000
```

Example using an internal service name:

```
AMMARE_MODEL_GATEWAY_URL=http://ammare-litellm:4000
```

In an early deployment, this endpoint may point directly to the local LLM service. In later deployments, it may point to LiteLLM or Headroom.

This allows the backend service chain to change while preserving the same application-level configuration pattern.

Proposed Service Inventory

The following service inventory describes the intended full topology.

Service	Purpose	Required Initially	Typical Port	Notes
ammare-langchain	Main middleware and orchestration layer	Yes	8080	Receives user/API requests and coordinates model/tool workflows.
ammare-local-llm	Local model inference endpoint	Yes	8000	Initial local model runtime, such as vLLM or similar.
ammare-lite-llm	Model routing layer	No	4000	Routes requests to local and cloud model providers.
ammare-headroom	Additional gateway/orchestration layer	No	4100	Added after core model routing is validated.
ammare-op-enhands	Coding-agent workspace service	No	TBD	Optional agentic coding environment.
ammare-vector-db	Vector storage for retrieval	No	TBD	Optional memory/retrieval backend.
ammare-embedding	Local embedding model endpoint	No	TBD	Optional service for document indexing and retrieval.
ammare-ui	Optional web UI or front-end	No	TBD	May be deferred depending on initial scope.

Ports marked **TBD** should be finalized during implementation and then updated in this document and the deployment documentation.

The coding agent should not invent hidden service names or undocumented ports. If a new service or port is required, it should update this topology section and the related documentation.

Required Initial Services

The minimum viable deployment should include:

ammare-langchain

```
ammare-local-llm
```

This supports the earliest working service chain:

```
User/API Client:  
→ ammare-langchain  
→ ammare-local-llm
```

This initial topology is intentionally simple. It validates that the middleware can reach a local model endpoint and return a response.

The initial deployment should still use a logical model gateway variable so the backend target can be changed later.

Optional Services

Optional services should be enabled through deployment configuration.

Optional components may include:

- LiteLLM.
- Headroom.
- OpenHands.
- Memory backend.
- Vector database.
- Embedding model.
- UI layer.
- Additional model providers.
- Additional management or observability components.

Optional services should not be required for the minimal local deployment.

If an optional service is disabled, dependent services should either bypass it cleanly or fail with a clear configuration error.

Service Naming Standard

Service names should be predictable and consistent.

Recommended naming pattern:

```
ammare-<component>
```

Examples:

```
ammare-langchain  
ammare-local-llm  
ammare-litellm  
ammare-headroom  
ammare-openhands  
ammare-vector-db  
ammare-embedding
```

Scripts, documentation, directories, environment variables, logs, and health checks should use these names consistently.

If the project later supports multiple instances of the same service type, the name should include a role or instance suffix.

Example:

```
ammare-local-llm-primary  
ammare-local-llm-secondary  
ammare-embedding-small  
ammare-embedding-large
```

Port Management

All service ports should be documented.

The project should maintain a central port table in the deployment documentation and, ideally, in a machine-readable configuration file.

The following endpoint and port reference table is the authoritative list of service endpoints, default ports, and network exposure. For the initial POC, the deployment host is 10.1.10.17 on

the 10.1.10.0/24 lab subnet; this address is a POC-specific reference only. Only the client-facing gateway endpoint (ammare-langchain) needs to be reachable from anywhere on the LAN subnet, because it is the endpoint that IDE and CLI clients connect to. All other services should bind only to host-local or container-internal networks and should not be published to the LAN.

Service	Default Port	Example Endpoint	Network Exposure
ammare-langchain	8080	http://10.1.10.17:8080	LAN-routable (10.1.10.0/24). Client-facing gateway; IDE/CLI clients connect here (AMMARE_GATEWAY_URL).
ammare-local-llm	8000	http://ammare-local-llm:8000	Internal only (host-local or container network).
ammare-litellm	4000	http://ammare-litellm:4000	Internal only (host-local or container network).
ammare-headroom	4100	http://ammare-headroom:4100	Internal only (host-local or container network).
ammare-openhands	TBD	http://ammare-openhands:TBD	Internal only by default; LAN exposure may be added at implementation if browser access from other machines is required.
ammare-vector-db	TBD	http://ammare-vector-db:TBD	Internal only (host-local or container network).
ammare-embedding	TBD	http://ammare-embedding:TBD	Internal only (host-local or container network).

ammare-ui	TBD	http://ammare-ui:TBD	Internal only by default; LAN exposure may be added at implementation if deployed for browser access.
-----------	-----	----------------------	---

The topology should distinguish between:

- Container internal ports.
- Host-published ports.
- Local-only ports.
- API ports.
- Health-check ports.
- Optional UI ports.

For local lab deployments, it is acceptable for services to bind to local lab IP addresses. Services that do not need external access should not be published beyond the host or internal container network.

The coding agent should avoid randomly selecting ports during implementation. New ports should be added deliberately and documented.

Network Model

The platform should support a local lab network model.

The initial deployment may use a single host with multiple containers. Later deployments may spread services across multiple hosts if required by GPU placement, resource constraints, or testing goals.

Supported network patterns may include:

- Single host with container-internal networking.
- Single host with selected host-published ports.
- Local lab IP-based service access.
- Internal DNS or static hostname mapping.
- Multi-host deployment using static IPs or local DNS.

The architecture should not assume public internet exposure.

Cloud model providers may require outbound internet access, but inbound public access should not be required for the core local platform.

Shared Configuration Paths

The deployment should use predictable configuration paths.

Recommended project-level paths:

```
config/  
env/  
scripts/  
docs/  
tests/  
logs/  
data/  
models/ (reference only, models should not be committed to git)
```

Recommended component-specific configuration paths:

```
config/langchain/  
config/litellm/  
config/headroom/  
config/local-llm/  
config/openhands/  
config/memory/
```

Recommended environment template file paths:

```
env/ammare.env.example  
env/langchain.env.example  
env/local-llm.env.example  
env/litellm.env.example  
env/headroom.env.example  
env/openhands.env.example  
env/memory.env.example
```

Recommended environment file paths (git ignored)

```
env/ammare.env  
env/langchain.env  
env/local-llm.env  
env/litellm.env  
env/headroom.env  
env/openhands.env  
env/memory.env
```

Actual environment files above that contain local values or secrets should not be committed, use the environment template files as examples and include them in repo commits

Note: All env files need to be well documented with inline comments explaining the usage of each defined variable.

Environment files are not hidden files. Actual environment files use the plain <component>.env naming convention and are excluded from version control through .gitignore, while the corresponding <component>.env.example templates are committed to the repository.

Shared Runtime Paths

Runtime data should be separated from static configuration.

Recommended runtime paths:

```
data/models/  
data/vector-db/  
data/openhands-workspaces/  
data/uploads/  
logs/  
tmp/
```

These paths should be documented so users know what persists across restarts and what can be safely removed.

The coding agent should not scatter persistent data into undocumented directories.

Environment Variable Standards

Environment variables should use a consistent prefix.

Recommended global prefix:

```
AMMARE_
```

Examples:

```
AMMARE_MODEL_GATEWAY_URL=  
AMMARE_LANGCHAIN_PORT=  
AMMARE_LITELLM_PORT=  
AMMARE_LOCAL_LLM_PORT=  
AMMARE_HEADROOM_PORT=  
AMMARE_DEPLOY_PROFILE=  
AMMARE_ENABLE_LITELLM=  
AMMARE_ENABLE_HEADROOM=  
AMMARE_ENABLE_OPENHANDS=  
AMMARE_ENABLE_MEMORY=
```

Component-specific variables may use the same prefix plus the component name.

Examples:

```
AMMARE_LANGCHAIN_LOG_LEVEL=  
AMMARE_LITELLM_CONFIG_PATH=  
AMMARE_LOCAL_LLM_MODEL_PATH=  
AMMARE_LOCAL_LLM_GPU_DEVICES=  
AMMARE_HEADROOM_UPSTREAM_URL=
```

Two gateway variables are intentionally distinct and must not be conflated:

AMMARE_GATEWAY_URL is the client-facing endpoint that IDE, CLI, and other end-user tools connect to (the ammare-langchain service, default port 8080), while

AMMARE_MODEL_GATEWAY_URL is the internal downstream model gateway that the

LangChain middleware sends model requests to (the local LLM endpoint, ammare-litellm, or ammare-headroom, depending on the active service chain).

The coding agent should prefer clear environment variables over hardcoded values.

All variables above should be created with a default value. End users can modify as needed.

Startup Order

The deployment should start services in dependency order.

A typical full startup order may be:

```
Local LLM Endpoint
LiteLLM
Headroom
Memory / Retrieval Services
OpenHands
LangChain Middleware
Optional UI
```

However, the exact order may vary depending on the enabled deployment profile.

The deployment scripts should validate each dependency before starting dependent services where practical.

For example:

- LiteLLM should validate that at least one model backend is configured.
- Headroom should validate that its upstream endpoint is reachable.
- LangChain should validate that its configured model gateway is reachable.
- OpenHands should validate that its workspace path exists.
- Memory services should validate that the vector database is available.

Independent Troubleshooting Paths

Each service should be independently testable.

The platform should provide scripts that can validate:

- Local LLM endpoint directly.
- LiteLLM to local LLM.
- LiteLLM to cloud provider.
- Headroom to LiteLLM.
- LangChain to configured model gateway.
- Full user-to-model service chain.
- Memory indexing.
- Memory retrieval.
- OpenHands workspace behavior.

This is important because the final chain may contain several layers. A failure in the full chain should be isolated quickly to the failing component.

Health Check Standard

Each service should provide or expose a health check.

Health checks should be lightweight and should not require a full model generation unless specifically testing inference behavior.

Recommended health-check categories:

- Container running.
- Port listening.
- HTTP health endpoint responding.
- Required config file present.
- Required upstream reachable.
- Required model loaded.
- Required volume mounted.
- Required credentials present if applicable.

The full validation script should combine these individual checks into a service-chain health report.

Logging Standard

Each service should write logs in a predictable way.

Logs should support both direct container log inspection and optional file-based collection.

Recommended logging expectations:

- Logs should identify service name.
- Logs should identify startup configuration summary without exposing secrets.
- Logs should show upstream endpoint selection.
- Logs should show request routing decisions where applicable.
- Logs should show validation failures clearly.
- Logs should not print API keys or credentials.
- Logs should be documented for each component.

The troubleshooting documentation should explain where to find logs for each service.

Deployment Profiles

The topology should support deployment profiles.

Example profiles:

```
minimal-local
local-with-litellm
hybrid-local-cloud
full-agentic
full-agentic-with-memory
```

Each profile should define which services are enabled and how the service chain is connected.

Example:

Profile	Enabled Services	Request Path
minimal-local	LangChain, Local LLM	LangChain → Local LLM
local-with-litellm	LangChain, LiteLLM, Local LLM	LangChain → LiteLLM → Local LLM
hybrid-local-cloud	LangChain, LiteLLM, Local LLM, Cloud Provider	LangChain → LiteLLM → Local / Cloud
full-agentic	LangChain, Headroom, LiteLLM, Local LLM, Cloud Provider, OpenHands	LangChain → Headroom → LiteLLM → Models

full-agentic-with-memory	LangChain, Headroom, LiteLLM, Models, OpenHands, Memory	LangChain → Headroom → LiteLLM → Models, with retrieval support
--------------------------	---	---

The deployment profile should be selected through configuration or a deployment flag.

Topology Update Requirement

This topology section should be updated whenever a service is added, removed, renamed, or rewired.

Changes that require topology updates include:

- New container service.
- New exposed port.
- New internal port.
- New volume mount.
- New environment file.
- New upstream dependency.
- New deployment profile.
- New health-check endpoint.
- New model provider.
- New service-chain path.

The coding agent should treat this section as a source of architectural truth. If implementation requires a topology change, the document should be updated in the same phase as the code change.

Summary

The aMMare target service topology defines the intended full deployment while still supporting phased implementation.

The initial system may start with only LangChain and a local LLM endpoint. Later phases can add LiteLLM, cloud providers, Headroom, OpenHands, memory, retrieval, and additional operational tooling.

The key design requirement is that service relationships remain configurable. IP addresses are acceptable for the local lab deployment, but service targets should still be expressed through logical configuration variables so the request path can evolve without rewriting core application logic.

Component Configuration Model

The aMMare platform should use a predictable and centralized configuration model.

Configuration should be externalized wherever practical so services can be added, removed, rewired, or replaced without requiring application code changes. This is especially important because the deployment will evolve across phases from a simple local model path to a more complete routed service chain.

The goal is to keep configuration clear, portable, and easy for both users and coding agents to modify safely.

Configuration Principles

Configuration should follow these principles:

- Avoid hardcoded service URLs in application code.
- Use environment variables for deployment-specific values.
- Use config files for structured routing and provider definitions.
- Keep example configuration files committed.
- Keep real secrets out of the repository.
- Validate configuration before deploying services.
- Keep global settings separate from component-specific settings.
- Document which files control each service.
- Use stable variable names even if the underlying service chain changes.

Configuration Layout

The repository should separate global configuration, component configuration, examples, and runtime data.

Recommended layout:

```
env/  
  ammare.env.example  
  langchain.env.example  
  local-llm.env.example
```

```
litellm.env.example  
headroom.env.example  
openhands.env.example  
memory.env.example
```

```
config/  
  models.yaml  
  routing.yaml  
  langchain/  
  local-llm/  
  litellm/  
  headroom/  
  openhands/  
  memory/
```

The **env/** directory should contain environment variable examples and local deployment values.

The **config/** directory should contain structured configuration files used by services and scripts.

Actual local environment files may be created from the examples but should not be committed if they contain host-specific values or secrets.

Global Configuration

Global configuration should define settings shared across the deployment.

Examples:

```
AMMARE_DEPLOY_PROFILE=  
AMMARE_BASE_DIR=  
AMMARE_CONFIG_DIR=  
AMMARE_DATA_DIR=  
AMMARE_LOG_DIR=  
AMMARE_ENABLE_LITELLM=  
AMMARE_ENABLE_HEADROOM=  
AMMARE_ENABLE_OPENHANDS=
```

```
AMMARE_ENABLE_MEMORY=  
AMMARE_LOG_LEVEL=
```

Global configuration should describe the active deployment profile and which optional components are enabled.

Component scripts should read these values rather than making independent assumptions about paths, profiles, or enabled services.

Component Configuration

Each component should have its own configuration file or environment file.

Examples:

```
env/langchain.env  
env/local-llm.env  
env/litellm.env  
env/headroom.env  
env/openhands.env  
env/memory.env
```

Each component configuration should define only the values required for that component.

For example, LangChain configuration may define:

```
AMMARE_LANGCHAIN_PORT=  
AMMARE_MODEL_GATEWAY_URL=  
AMMARE_LANGCHAIN_LOG_LEVEL=  
AMMARE_LANGCHAIN_RULES_PATH=
```

Local LLM configuration may define:

```
AMMARE_LOCAL_LLM_PORT=  
AMMARE_LOCAL_LLM_MODEL_PATH=
```

```
AMMARE_LOCAL_LLM_GPU_DEVICES=
AMMARE_LOCAL_LLM_RUNTIME=
```

LiteLLM configuration may define:

```
AMMARE_LITELLM_PORT=
AMMARE_LITELLM_CONFIG_PATH=
AMMARE_LITELLM_DEFAULT_MODEL=
```

Headroom configuration may define:

```
AMMARE_HEADROOM_PORT=
AMMARE_HEADROOM_UPSTREAM_URL=
AMMARE_HEADROOM_CONFIG_PATH=
```

The coding agent should not duplicate the same setting across multiple files unless there is a clear reason.

Logical Endpoint Configuration

Service-to-service communication should use logical endpoint variables.

The most important example is the model gateway endpoint:

```
AMMARE_MODEL_GATEWAY_URL=
```

This value may point to a local IP address, a container service name, local DNS, LiteLLM, Headroom, or another model gateway layer.

Examples:

```
AMMARE_MODEL_GATEWAY_URL=http://10.1.10.17:8000
AMMARE_MODEL_GATEWAY_URL=http://10.1.10.17:4000
```

```
AMMARE_MODEL_GATEWAY_URL=http://ammare-litellm:4000  
AMMARE_MODEL_GATEWAY_URL=http://ammare-headroom:4100
```

The value can change between phases, but LangChain should continue reading the same variable.

This allows the service chain to evolve without rewriting LangChain application logic.

Model Endpoint Registry

Model endpoints should be defined in a structured registry.

Recommended file:

```
config/models.yaml
```

This file should describe available local and cloud model endpoints.

Example structure:

```
models:  
  local_default:  
    type: local  
    provider: vllm  
    base_url: http://10.1.10.17:8000  
    model: local-model-name  
    enabled: true  
  
  cloud_default:  
    type: cloud  
    provider: openai-compatible  
    base_url: https://provider.example.com/v1  
    model: cloud-model-name  
    enabled: false
```

The registry should store model metadata and endpoint definitions, but it should not store raw API keys.

Routing Configuration

Routing rules should be stored separately from application code where practical.

Recommended file:

```
config/routing.yaml
```

This file may define:

- Default model.
- Local-first behavior.
- Model aliases.
- Escalation targets.
- Fallback targets.
- Task-to-model mappings.
- Risk-based routing behavior.

Example structure:

```
default_model: local_default

routing:
  simple:
    model: local_default
  coding:
    model: local_coding
    fallback: cloud_default
  high_reasoning:
    model: cloud_default

escalation:
  enabled: true
  fallback_on_failure: true
```

The routing file should be simple at first and expanded only as routing behavior becomes more advanced.

Secrets Handling

Secrets should be kept separate from committed configuration.

Secrets may include:

- Cloud model API keys.
- Service tokens.
- Webhook credentials.
- Repository credentials.
- Private endpoints.
- Authentication tokens.

Recommended handling:

```
env/secrets.env
```

or a deployment-specific secrets mechanism.

Secret files should be excluded from version control.

Example:

```
OPENAI_API_KEY=  
ANTHROPIC_API_KEY=  
CLOUD_MODEL_API_KEY=
```

Application logs, validation scripts, and error messages should not print secret values.

Configuration Validation

The project should include configuration validation scripts.

Recommended script:

```
scripts/validate-config.sh
```

This script should check:

- Required environment files exist.
- Required variables are set.
- Enabled services have required configuration.
- Ports are valid.
- URLs are valid.
- Referenced files and directories exist.
- Model registry entries are valid.
- Routing rules reference known models.
- Secret variables are present when cloud providers are enabled.

Configuration validation should run before deployment and as part of the full service validation process.

Configuration Ownership

Each configuration file should have a clear purpose.

Suggested ownership:

File	Purpose
env/ammare.env	Global deployment settings.
env/<component>.env	Component-specific environment variables.
env/secrets.env	Local secrets, not committed.
config/models.yaml	Model endpoint registry.
config/routing.yaml	Model routing and escalation rules.
config/<component>/	Component-specific structured config.

The coding agent should update the correct file rather than creating new configuration locations without reason.

Summary

The aMMare configuration model should keep deployment-specific values, component settings, model definitions, routing rules, and secrets clearly separated.

The most important rule is that service relationships must remain configurable. Local IP addresses are acceptable for the lab deployment, but they should still be assigned through environment variables or structured config files rather than hardcoded into application logic.

Service Chaining and Rewiring Strategy

The aMMare service chain will evolve over multiple phases.

Early deployments may connect LangChain directly to a local model endpoint. Later deployments may introduce LiteLLM, cloud model providers, Headroom, memory, retrieval, or coding-agent services. The architecture must support these changes without requiring major rewrites to the LangChain layer or deployment scripts.

The goal is to make service chaining intentional, configurable, and testable.

Chaining Principle

Each service should connect to the next layer through configuration, not hardcoded application logic.

A service should not assume that its upstream or downstream target will always be the same component. This is especially important for LangChain, because the model path may change several times during development.

The model path may evolve from:

```
LangChain → Local LLM
```

to:

```
LangChain → LiteLLM → Local LLM
```

to:

```
LangChain → LiteLLM → Local LLM / Cloud LLM
```

to:

```
LangChain → Headroom → LiteLLM → Local LLM / Cloud LLM
```

Each transition should be handled by configuration changes and validation scripts whenever practical.

Logical Gateway Pattern

LangChain should use a logical model gateway endpoint.

Recommended variable:

```
AMMARE_MODEL_GATEWAY_URL=
```

This value should represent the next service in the active model path.

Examples:

```
AMMARE_MODEL_GATEWAY_URL=http://10.1.10.17:8000  
AMMARE_MODEL_GATEWAY_URL=http://10.1.10.17:4000  
AMMARE_MODEL_GATEWAY_URL=http://ammare-litellm:4000  
AMMARE_MODEL_GATEWAY_URL=http://ammare-headroom:4100
```

The value may be a local IP address, local DNS name, container service name, or reverse-proxy name. For the initial lab deployment, static local IPs are acceptable.

The important requirement is that LangChain reads this value from configuration and does not hardcode a specific backend.

Phase-Based Chain Evolution

The service chain should change only after the current chain is validated.

Recommended progression:

Note: the stages below describe the evolution of the service chain and are numbered independently of the Development and Deployment Roadmap phases.

Stage 1:

```
LangChain → Local LLM
```

Stage 2:

```
LangChain → LiteLLM → Local LLM
```

Stage 3:

```
LangChain → LiteLLM → Local LLM / Cloud LLM
```

Stage 4:

```
LangChain → Headroom → LiteLLM → Local LLM / Cloud LLM
```

Stage 5:

```
LangChain → Headroom → LiteLLM → Local LLM / Cloud LLM
    ↘ Memory / Retrieval
    ↘ OpenHands / Coding Workspace
```

The coding agent should not add multiple new chain layers in the same step unless explicitly instructed.

Each phase should include service-specific validation and full-chain validation before moving forward.

Rewiring Events

A rewiring event occurs when one service is inserted, removed, bypassed, or replaced in the request path.

Examples:

- Repointing LangChain from the local LLM to LiteLLM.
- Repointing LangChain from LiteLLM to Headroom.
- Adding a cloud model backend to LiteLLM.
- Temporarily bypassing Headroom for troubleshooting.
- Disabling memory or retrieval.
- Replacing a local model endpoint.
- Moving a service from one host IP to another.

Rewiring should be treated as an intentional change. It should update configuration, documentation, and validation scripts in the same phase.

Rewiring Requirements

Each rewiring event should include:

- Updated environment variables.
- Updated service configuration files.
- Updated deployment profile if needed.
- Updated topology documentation.
- Updated validation script behavior.
- Updated troubleshooting notes.
- Confirmation that the previous layer can still be tested independently.
- Confirmation that the new full chain works end to end.

The coding agent should not silently change service paths without updating the related documentation.

Bypass Paths

The architecture should support bypass paths for troubleshooting.

Bypass paths allow a user to isolate a failing component without dismantling the whole deployment.

Examples:

```
LangChain → Local LLM
```

```
LangChain → LiteLLM → Local LLM
```

```
LangChain → Headroom → LiteLLM → Local LLM
```

```
LiteLLM → Local LLM
```

```
LiteLLM → Cloud LLM
```

Each major service should have a direct validation script so failures can be isolated quickly.

Example scripts:

```
scripts/test-local-llm.sh
scripts/test-litellm-local.sh
scripts/test-litellm-cloud.sh
scripts/test-headroom.sh
scripts/test-langchain-gateway.sh
scripts/test-full-chain.sh
```

Deployment Profiles

Deployment profiles should define which chain is active.

Examples:

```
minimal-local  
local-with-litellm  
hybrid-local-cloud  
full-agentic  
full-agentic-with-memory
```

Each profile should explicitly define:

- Enabled services.
- Disabled services.
- LangChain gateway target.
- Model routing layer.
- Active model providers.
- Whether Headroom is in path.
- Whether memory/retrieval is enabled.
- Whether OpenHands is enabled.

A deployment profile should make the active chain obvious without requiring users to inspect multiple files.

Avoiding Early Hardcoding

The initial implementation should avoid shortcuts that make later rewiring difficult.

The coding agent should avoid:

- Hardcoding LangChain directly to a local model URL.
- Embedding provider-specific model names throughout application logic.
- Duplicating endpoint URLs in multiple scripts.
- Assuming LiteLLM is always present.
- Assuming Headroom is always present.
- Assuming cloud model providers are always available.
- Assuming memory is always enabled.
- Assuming all services run on the same host.

- Selecting undocumented ports.
- Creating undocumented config files.

Temporary direct connections are acceptable during early phases, but they must still be expressed through configuration.

Service Contract Expectations

Each service boundary should have a basic contract.

At minimum, each service should define:

- Input API or request format.
- Output format.
- Health-check behavior.
- Error behavior.
- Required environment variables.
- Required upstream services.
- Expected downstream services.
- Timeout behavior.
- Logging behavior.

This helps prevent later components from depending on unclear or accidental behavior.

For example, LangChain should know only that it has a configured model gateway. It should not need to know whether that gateway is the local model endpoint, LiteLLM, or Headroom.

Configuration-Driven Chain Selection

The active service chain should be selected by configuration.

Recommended variables:

```
AMMARE_DEPLOY_PROFILE=  
AMMARE_MODEL_GATEWAY_URL=  
AMMARE_ENABLE_LITELLM=  
AMMARE_ENABLE_HEADROOM=  
AMMARE_ENABLE_MEMORY=  
AMMARE_ENABLE_OPENHANDS=
```

The top-level deployment script should read these values and deploy only the selected services.

Validation scripts should also read the active profile so they test the correct chain.

Validation After Rewiring

Every service-chain change should be validated.

Validation should include:

- Confirm enabled services are running.
- Confirm required ports are listening.
- Confirm configured URLs are reachable.
- Confirm each service can reach its upstream.
- Confirm each service can reach its downstream.
- Confirm a test prompt succeeds through the active chain.
- Confirm logs show the expected path.
- Confirm disabled services are not required by the active profile.

The validation result should identify where the chain fails if a test does not pass.

Failure Isolation

The service chain should be designed for fast failure isolation.

When a full-chain request fails, the troubleshooting path should move from downstream to upstream:

```
Local LLM
LiteLLM
Headroom
LangChain
User/API Client
```

For example:

- Test the local model directly.
- Test LiteLLM against the local model.
- Test LiteLLM against any cloud provider.

- Test Headroom against LiteLLM.
- Test LangChain against the configured gateway.
- Test the full user-facing chain.

This avoids guessing which component is failing.

Documentation Requirement

Every service-chain mode should be documented.

Documentation should include:

- Active request path.
- Required services.
- Required ports.
- Required environment variables.
- How to validate the chain.
- How to bypass each optional layer.
- Common failure points.
- How to revert to the previous chain.

This is especially important because the project will be built in phases and may be used by a coding agent during implementation.

Summary

The aMMare service chain must be able to evolve without forcing major rewrites.

The core strategy is to use logical endpoints, external configuration, deployment profiles, and validation scripts. Early phases may connect LangChain directly to a local model, but the implementation should preserve the ability to insert LiteLLM, cloud providers, Headroom, memory, retrieval, and OpenHands later.

Each rewiring event should be deliberate, documented, and validated before additional complexity is added.

One-Click Deployment and Modular Install Strategy

The aMMare platform should support a one-click deployment model while preserving modular component control.

The top-level deployment process should make the system easy to install, but it should not hide all logic inside a single large script. The preferred model is a top-level orchestration script that calls component-specific deployment scripts in the correct order.

This keeps deployment simple for the user while keeping the implementation maintainable for future changes.

Deployment Principle

The deployment process should be:

- Modular.
- Idempotent.
- Profile-driven.
- Configurable.
- Validated after each major step.
- Safe to rerun.
- Easy to troubleshoot.

The top-level deployment script should coordinate the process. Component scripts should own the details of deploying each service.

Top-Level Deployment Script

The main deployment entry point should be:

```
scripts/deploy.sh
```

This script should:

- Load global configuration.
- Detect the selected deployment profile.

- Run preflight checks.
- Prompt to install pre-requisites (if not found)
- Determine which components are enabled.
- Deploy components in the correct order.
- Call validation scripts after deployment.
- Print a clear deployment summary.
- Stop with a useful error if a required step fails.
- Log deployment operations to a rotating log file

The top-level script should not contain all component-specific logic directly.

Component Deployment Scripts

Each major service should have its own deployment script.

Recommended examples:

```
scripts/deploy-local-llm.sh
scripts/deploy-litellm.sh
scripts/deploy-headroom.sh
scripts/deploy-langchain.sh
scripts/deploy-openhands.sh
scripts/deploy-memory.sh
```

Each component script should be responsible for:

- Loading its component configuration.
- Checking required variables.
- Creating required directories.
- Creating or updating service definitions.
- Starting or restarting the service.
- Running a component-level validation check.
- Reporting success or failure clearly.

This allows components to be deployed independently during development and troubleshooting.

Deployment Profiles

Deployment profiles should define which components are deployed and how they are connected.

Example profiles:

```
minimal-local  
local-with-litellm  
hybrid-local-cloud  
full-agentic  
full-agentic-with-memory
```

The active profile should be selected through configuration or a deployment flag.

Example:

```
AMMARE_DEPLOY_PROFILE=minimal-local
```

The deployment profile should control:

- Enabled services.
- Disabled services.
- Service startup order.
- Active model gateway target.
- Optional routing layers.
- Optional memory and retrieval services.
- Optional coding-agent workspace services.

A profile should make the active deployment mode clear without requiring users to inspect multiple scripts.

Deployment Order

Services should be deployed in dependency order.

A typical full deployment order is:

```
Local LLM
LiteLLM
Headroom
Memory / Retrieval
OpenHands
LangChain
Optional UI (and/or configuration to use deployed pipeline)
```

Minimal deployments may skip most of these services.

The deployment script should not assume every component is enabled. It should read the active profile and deploy only the required components.

Preflight Checks

Before deploying services, the top-level script should run preflight validation.

Preflight checks should confirm:

- Required tools are installed.
- Required directories exist or can be created.
- Required environment files exist.
- Required variables are set.
- Required ports are available.
- Required model paths exist.
- Required GPU devices are visible if needed.
- Required secrets are present for enabled cloud providers.
- The selected deployment profile is valid.

Preflight failures should stop deployment before services are modified.

Idempotency Requirements

Deployment scripts should be safe to rerun.

Scripts should avoid assuming a clean system. They should check current state before making changes.

Examples:

- Create directories only if missing.
- Update service files predictably.
- Restart only the services that need restarting.
- Do not duplicate environment entries.
- Do not overwrite local configuration without warning.
- Do not delete persistent data during normal deployment.
- Report when no change is required.

This allows users and coding agents to re-run deployment after changes without rebuilding everything manually.

Optional Component Flags

The deployment system should allow optional components to be enabled or disabled.

Examples:

```
AMMARE_ENABLE_LITELLM=true  
AMMARE_ENABLE_HEADROOM=false  
AMMARE_ENABLE_OPENHANDS=false  
AMMARE_ENABLE_MEMORY=false
```

The deployment script should respect these flags.

If a component is disabled, the deployment should either bypass it cleanly or report a clear configuration conflict if another enabled component depends on it.

Validation During Deployment

Deployment should include validation at multiple levels.

Recommended validation stages:

- Preflight validation.
- Component-level validation after each service is deployed.
- Service-chain validation after related services are connected.
- Final deployment validation.

Example scripts:

```
scripts/validate-config.sh
scripts/validate-local-llm.sh
scripts/validate-litellm.sh
scripts/validate-headroom.sh
scripts/validate-langchain.sh
scripts/validate-service-chain.sh
```

The final deployment result should clearly state which components passed, failed, or were skipped.

Uninstall and Reset Behavior

The project should include controlled uninstall behavior.

Recommended script:

```
scripts/uninstall.sh
```

The uninstall process should distinguish between:

- Removing containers or services.
- Removing generated service definitions.
- Removing temporary files.
- Preserving persistent data.
- Removing persistent data only when explicitly requested.

Normal uninstall should not delete models, vector databases, workspaces, logs, or user configuration unless the user explicitly requests a destructive reset.

A destructive reset should require a clear flag such as:

```
--purge-data
```

Generated Files

Deployment scripts may generate runtime files, service definitions, or local environment files.

Generated files should be predictable and documented.

The project should define:

- Which files are generated.
- Which files are user-edited.
- Which files are safe to delete.
- Which files should not be committed.
- Which files are examples only.

The coding agent should avoid creating undocumented generated files.

Deployment Summary Output

At the end of deployment, the script should print a clear summary.

The summary should include:

- Selected deployment profile.
- Enabled components.
- Skipped components.
- Service names.
- Listening ports.
- Active model gateway URL.
- Validation result.
- Log locations.
- Next suggested command.

Example summary fields:

```
Deployment profile: minimal-local
Enabled services: ammare-langchain, ammare-local-llm
Model gateway: http://10.1.10.17:8000
Validation: passed
Next command: scripts/validate-service-chain.sh
```

The summary should help the user understand what was deployed without reading logs.

Coding Agent Expectations

The coding agent should implement deployment incrementally.

For each deployment phase, the coding agent should:

- Add or update only the scripts required for that phase.
- Keep top-level orchestration simple.
- Keep component logic in component scripts.
- Add validation alongside deployment.
- Update documentation when deployment behavior changes.
- Avoid hardcoded paths, ports, and service URLs.
- Preserve existing working profiles unless explicitly changing them.

The coding agent should not build a full deployment framework before the early components are working.

Summary

The aMMare deployment strategy should provide a simple top-level deployment experience while preserving modular control underneath.

The user should be able to run one command for a selected deployment profile, but each service should still have its own deploy and validation logic. This approach supports phased development, easier troubleshooting, selective component deployment, and safer long-term maintenance.

Helper Scripts and Operational Tooling

The aMMare platform should include helper scripts that make deployment, validation, troubleshooting, and ongoing management easier.

These scripts are intended for both human users and coding agents. They should provide fast ways to confirm service health, isolate failures, inspect configuration, and manage model endpoints without requiring manual command discovery.

Tooling Principles

Helper scripts should follow these principles:

- Be simple to run.
- Produce clear pass/fail output.
- Avoid hiding important errors.
- Avoid requiring users to remember long commands.
- Support both individual service checks and full-chain checks.
- Read configuration from the same files used by deployment.
- Avoid hardcoded paths, ports, and URLs.
- Avoid printing secrets.
- Be safe to run repeatedly.

The scripts should support daily operation, not only first-time deployment.

Recommended Script Categories

The project should include helper scripts in the following categories:

Category	Purpose
Configuration checks	Validate environment files, config files, ports, paths, and enabled services.
Component health checks	Confirm each individual service is running and reachable.
Service-chain validation	Confirm requests can move through the active chain.
Model management	List, add, remove, and validate local or cloud model endpoints.
Log inspection	Show relevant logs for one service or the active chain.
Troubleshooting bundles	Collect useful diagnostic output for review.
Lifecycle management	Start, stop, restart, deploy, and uninstall services safely.

Core Validation Scripts

The following scripts should exist early in the project:

```
scripts/validate-config.sh
scripts/validate-local-llm.sh
scripts/validate-langchain.sh
scripts/validate-service-chain.sh
```

As components are added, additional validation scripts should be created:

```
scripts/validate-litellm.sh
scripts/validate-headroom.sh
scripts/validate-openhands.sh
scripts/validate-memory.sh
scripts/validate-model-registry.sh
scripts/validate-routing.sh
```

Each validation script should return a non-zero exit code on failure so it can be used by deployment scripts and CI workflows.

Service-Chain Validation

The service-chain validation script should test the active deployment profile.

Recommended script:

```
scripts/validate-service-chain.sh
```

This script should:

- Load the active deployment profile.
- Identify enabled services.
- Confirm required ports are reachable.
- Confirm required health endpoints respond.
- Send a basic test request through the active chain.
- Report which layer failed if validation does not pass.

The script should not assume the full deployment is always enabled. It should validate the chain that matches the current profile.

Endpoint Testing Scripts

Endpoint testing scripts should isolate individual service boundaries.

Recommended scripts:

```
scripts/test-local-llm.sh
scripts/test-langchain-gateway.sh
scripts/test-litellm-local.sh
scripts/test-litellm-cloud.sh
scripts/test-headroom.sh
scripts/test-full-chain.sh
```

These scripts should make it easy to determine whether a problem is with the local model, routing layer, Headroom, LangChain, or the full chain.

Model Management Scripts

The project should provide helper scripts for managing model endpoints.

Recommended scripts:

```
scripts/list-models.sh
scripts/add-model-endpoint.sh
scripts/remove-model-endpoint.sh
scripts/test-model-endpoint.sh
```

These scripts should work against the model endpoint registry, such as:

```
config/models.yaml
```

They should help users add new local models, cloud models, or provider endpoints without manually editing multiple files.

When a model is added, the scripts should identify whether routing configuration also needs to be updated.

Routing Management Scripts

Routing should be inspectable and testable.

Recommended scripts:

```
scripts/show-routing.sh
scripts/test-routing.sh
scripts/validate-routing.sh
```

These scripts should show:

- Active default model.
- Available model aliases.
- Enabled escalation paths.
- Fallback models.
- Task-to-model mappings.
- Whether referenced models exist in the model registry.

This helps users and coding agents confirm that routing behavior matches the intended configuration.

Log Inspection Scripts

The project should include scripts for common log checks.

Recommended scripts:

```
scripts/logs-langchain.sh
scripts/logs-local-llm.sh
scripts/logs-litellm.sh
scripts/logs-headroom.sh
scripts/logs-service-chain.sh
```

These scripts should collect relevant logs without requiring users to remember container names or runtime commands.

Log scripts should redact secrets and avoid dumping excessive output by default.

Support Bundle Script

A support bundle script should collect useful diagnostic information.

Recommended script:

```
scripts/generate-support-bundle.sh
```

The support bundle may include:

- Deployment profile.
- Enabled services.
- Service status.
- Listening ports.
- Redacted environment summary.
- Config validation output.
- Recent service logs.
- Model registry summary.
- Routing summary.
- Service-chain validation result.

The bundle should not include raw secrets, private keys, or full credential files.

Service Lifecycle Scripts

The project should provide simple lifecycle scripts.

Recommended scripts:

```
scripts/ammare-start.sh  
scripts/ammare-stop.sh  
scripts/ammare-restart.sh
```

```
scripts/ammare-status.sh
```

These scripts should operate against the active deployment profile by default.

They may also support targeting a specific service:

```
scripts/ammare-restart.sh langchain  
scripts/ammare-status.sh litellm
```

Service lifecycle scripts should use the documented service names and should not require users to know the underlying container runtime commands.

Coding Agent Support

Helper scripts should make it easier for a coding agent to work safely.

The coding agent should use validation scripts instead of inventing ad hoc test commands whenever possible.

Expected coding-agent workflow:

```
scripts/validate-config.sh  
scripts/validate-<component>.sh  
scripts/validate-service-chain.sh
```

When the coding agent adds or modifies a component, it should also update or create the matching helper script.

If a new service is added without a validation script, the implementation should be considered incomplete.

Output Standards

Helper scripts should use consistent output formatting.

Recommended statuses:

```
PASS  
FAIL  
WARN  
SKIP  
INFO
```

Each script should clearly report:

- What was checked.
- What passed.
- What failed.
- What was skipped.
- What command or file should be reviewed next.

Scripts should return appropriate exit codes:

```
0 = success  
1 = failure  
2 = invalid usage or missing configuration
```

Documentation Requirement

Each helper script should be documented.

Documentation should include:

- Purpose.
- Usage examples.
- Required configuration.
- Expected output.
- Common failures.
- Related scripts.

The main README should list the most important scripts. Component-specific documentation should reference the scripts relevant to that component.

Summary

Helper scripts are a required part of the aMMare architecture.

They provide repeatable validation, simplify troubleshooting, support modular deployment, and give coding agents a safe way to verify changes. Every major service should have deployment, validation, logging, and troubleshooting support so the system remains manageable as the service chain grows.

Documentation Strategy

The aMMare project should include documentation for both human users and coding agents.

The documentation should explain the architecture, deployment model, component behavior, configuration files, validation scripts, and safe modification procedures. It should also provide enough implementation guidance for a coding agent to work in phases without losing the overall design intent.

Documentation Principles

Documentation should follow these principles:

- Keep the main README concise and practical.
- Put detailed component documentation in separate files.
- Document how to deploy, validate, troubleshoot, and modify each component.
- Keep architecture decisions separate from step-by-step operations.
- Update documentation in the same phase as code changes.
- Avoid duplicating the same explanation across multiple files.
- Clearly identify optional components and deferred features.
- Include examples where they reduce ambiguity.
- Treat documentation as part of the deliverable, not an afterthought.

Recommended Documentation Layout

Recommended structure:

```
README.md
```

```
docs/
```

```
architecture.md
development-roadmap.md
deployment.md
configuration.md
service-topology.md
service-chaining.md
validation.md
troubleshooting.md
coding-agent-implementation-guide.md

docs/components/
  langchain.md
  local-llm.md
  litellm.md
  headroom.md
  openhands.md
  memory.md
  models.md
  routing.md
```

This structure keeps the main README readable while allowing deeper documentation where needed.

Main README

The main [README.md](#) should be the user's starting point.

It should include:

- Project purpose.
- High-level architecture summary.
- Supported deployment profiles.
- Basic prerequisites.
- Minimal deployment instructions.
- Common validation commands.
- Documentation index.
- Known limitations.
- Current implementation status.

The README should not try to contain every technical detail. It should point to the correct detailed document when more information is needed.

However, always assume more documentation is needed, but do not assume that you should link directly from the [README.md](#). Rather assume that you will be documenting topics related to an already documented components and add new and additional supporting documents to the appropriate md file linked from the [README.md](#)

Architecture Documentation

The architecture document should describe the intended system design.

It should explain:

- Core architecture goals.
- Service chain.
- Model routing strategy.
- Tool execution and safety model.
- Configuration model.
- Deployment model.
- Validation strategy.
- Deferred features.

This document should remain conceptual and design-focused. Detailed commands should live in deployment, validation, or component documents.

Component Documentation

Each major component should have its own documentation file.

Each component document should include:

- Purpose of the component.
- Role in the service chain.
- Required configuration files.
- Required environment variables.
- Ports and endpoints.
- Deployment script.
- Validation script.

- Log inspection script.
- Common troubleshooting steps.
- Safe modification instructions.

Component documentation should be updated whenever the component's behavior, configuration, ports, or service-chain role changes.

Modification Guides

The project should document how to modify each major layer.

Examples:

- How to change LangChain prompts or rules.
- How to modify LangChain routing logic.
- How to add or remove LangChain tools.
- How to change the local LLM model.
- How to add a model endpoint to LiteLLM.
- How to add a cloud model provider.
- How to update routing and escalation rules.
- How to enable or disable Headroom.
- How to enable or disable memory.
- How to validate changes after modification.

These guides are important because the system is expected to evolve after the initial deployment.

Deployment Documentation

Deployment documentation should explain how to install and operate the platform.

It should include:

- Supported deployment profiles.
- Required host assumptions.
- Required tools.
- Required directories.
- Environment file setup.
- One-click deployment command.
- Component-level deployment commands.
- Uninstall behavior.

- Reset or purge behavior.
- Post-deployment validation.

Deployment documentation should match the actual scripts.

If the deployment process changes, the documentation must be updated in the same phase.

Configuration Documentation

Configuration documentation should explain which files control which behavior.

It should include:

- Global environment file.
- Component environment files.
- Model registry.
- Routing configuration.
- Secrets handling.
- Deployment profiles.
- Example local-lab configuration.
- Example cloud-provider configuration.
- Configuration validation command.

This documentation should make it clear where a user or coding agent should make changes.

Validation and Troubleshooting Documentation

Validation documentation should explain how to prove the system is working.

It should include:

- Configuration validation.
- Component validation.
- Endpoint validation.
- Service-chain validation.
- Routing validation.
- Model endpoint validation.
- Memory/retrieval validation if enabled.
- Expected pass/fail output.

Troubleshooting documentation should provide failure isolation paths.

Example flow:

```
Test Local LLM
Test LiteLLM
Test Headroom
Test LangChain
Test full service chain
```

Troubleshooting documentation should reference helper scripts instead of requiring users to assemble long manual commands.

Coding Agent Documentation

The coding agent implementation guide should provide explicit development instructions.

It should include:

- Build phases.
- File layout expectations.
- Script standards.
- Configuration rules.
- Validation requirements.
- Documentation update requirements.
- What not to hardcode.
- How to stop at phase boundaries.
- How to report completed work.

This document should be written as an operating guide for a coding agent that is modifying the repository incrementally.

Documentation Update Requirement

Documentation must be updated whenever implementation changes affect user behavior, service behavior, or architecture assumptions.

Documentation updates are required when:

- A service is added or removed.
- A port changes.

- A configuration file changes.
- An environment variable is added or renamed.
- A deployment profile changes.
- A service-chain path changes.
- A helper script is added or modified.
- A validation step changes.
- A troubleshooting process changes.
- A deferred feature becomes active.

A code change that modifies behavior but leaves documentation stale should be considered incomplete.

Avoiding Redundancy

The documentation set should avoid repeating the same detailed content in multiple files.

Recommended ownership:

Content Type	Primary Location
Quick start	README.md
Architecture decisions	docs/architecture.md
Deployment steps	docs/deployment.md
Configuration files	docs/configuration.md
Service names and ports	docs/service-topology.md
Component behavior	docs/components/<component>.md
Troubleshooting	docs/troubleshooting.md
Coding-agent instructions	docs/coding-agent-implementation-guide.md

Other documents may reference these sections, but they should not duplicate them in full.

Summary

The aMMare documentation strategy should support both understanding and implementation.

The README should provide a practical entry point. Detailed documentation should live under [docs/](#), with separate files for architecture, deployment, configuration, validation, troubleshooting, and individual components. Documentation should be updated alongside code so the project remains usable by both human operators and coding agents.

Coding Agent Implementation Guide

The aMMare architecture document should also serve as planning input for the coding agent.

The coding agent is expected to build the platform incrementally, validate each phase, and update documentation as implementation changes are made. It should not attempt to build the complete architecture in one large pass.

Implementation Principles

The coding agent should follow these principles:

- Build in phases.
- Make small, reviewable changes.
- Validate each change before moving on.
- Avoid hardcoded paths, ports, service URLs, and model names.
- Use the documented configuration model.
- Use the documented service names where possible.
- Keep deployment scripts modular.
- Keep helper scripts simple and repeatable.
- Update documentation with code changes.
- Stop at phase boundaries unless instructed to continue.
- Commit changes or prompt end-user to commit.

The coding agent should treat this architecture document as the source of intent, but it should still inspect the current repository state before making changes.

Phase-Based Development

The coding agent should work through the roadmap one phase at a time.

For each phase, it should:

- Review the phase objective.
- Identify required files.

- Identify affected services.
- Identify required configuration.
- Create or update only the files needed for that phase.
- Add validation for the new behavior.
- Run available validation.
- Update relevant documentation.
- Summarize what changed.
- Identify what remains deferred.

A phase is not complete until the related deployment, validation, and documentation are updated.

Any changes that are required to fix broken components should be reviewed in order to ensure that the configuration change does not alter the functionality of any existing component of the POC.

If changes are needed that affect the architecture as a whole, there must be a human-in-the-loop (HITL) review, and documentation should be reviewed to ensure that any such changes are well documented.

Required Work Pattern

For each implementation task, the coding agent should use this pattern:

```
Inspect → Plan → Modify → Validate → Document → Report
```

The coding agent should not begin by rewriting large parts of the repository.

It should first inspect the current files and understand what already exists. Then it should propose or follow a limited plan, make the smallest useful changes, run validation, update documentation, and report the result.

Repository Awareness

Before modifying files, the coding agent should inspect the repository structure.

It should identify:

- Existing scripts.

- Existing configuration files.
- Existing documentation.
- Existing container or service definitions.
- Existing tests.
- Existing examples.
- Existing naming conventions.

The coding agent should extend existing patterns when they are reasonable instead of creating parallel structures.

If existing files conflict with this architecture document, the coding agent should flag the conflict before making broad changes.

Configuration Rules

The coding agent should use the documented configuration model.

It should not hardcode:

- Service URLs.
- Local IP addresses.
- Ports.
- Model names.
- Model paths.
- API keys.
- Secrets.
- Deployment profiles.
- Runtime directories.

These values should come from environment files, structured config files, or documented defaults.

Local IP addresses are acceptable for the lab deployment, but they must still be assigned through configuration.

Service Chain Rules

The coding agent should preserve the ability to rewire the service chain.

LangChain should not be permanently tied to one backend model endpoint. It should use a logical gateway value such as:

```
AMMARE_MODEL_GATEWAY_URL=
```

When LiteLLM, Headroom, or another gateway layer is added, the coding agent should update configuration and validation rather than rewriting LangChain unnecessarily.

Service-chain changes should update:

- Environment files.
- Deployment profiles.
- Validation scripts.
- Topology documentation.
- Troubleshooting documentation.

Script Standards

Scripts should be written for repeatable operations.

Deployment, validation, troubleshooting, and lifecycle scripts should:

- Be safe to rerun.
- Use clear names.
- Read shared configuration.
- Print clear status output.
- Return useful exit codes.
- Avoid destructive actions by default.
- Avoid printing secrets.
- Fail clearly when required configuration is missing.
- Color code status codes for additional clarity.

Recommended status labels:

```
PASS  
FAIL  
WARN  
SKIP  
INFO
```

The coding agent should not create one-off scripts without documenting their purpose.

Deployment Script Expectations

The top-level deployment script should orchestrate component scripts.

The coding agent should keep this separation:

```
scripts/deploy.sh  
  calls component deployment scripts
```

```
scripts/deploy-<component>.sh  
  deploys one component
```

The top-level script should not become a large monolithic script containing all component logic.

When a new component is added, the coding agent should add or update:

- Component deployment script.
- Component validation script.
- Component environment example.
- Component documentation.
- Top-level deployment integration if appropriate.

Validation Requirements

Every meaningful implementation step should include validation.

Validation may include:

- Shell syntax checks.
- Configuration validation.
- Service health checks.
- Endpoint tests.
- Service-chain tests.
- Routing tests.

- Model endpoint tests.
- Documentation link checks if available.

The coding agent should prefer existing validation scripts over ad hoc commands.

If no validation script exists for a new component, the coding agent should create one.

Documentation Requirements

The coding agent should update documentation when it changes behavior.

Documentation updates are required when:

- A new service is added.
- A new port is introduced.
- A config variable is added or changed.
- A deployment profile changes.
- A script is added or changed.
- A service-chain path changes.
- A troubleshooting process changes.
- A model endpoint process changes.
- A deferred feature becomes active.

Documentation should be updated in the same implementation phase as the code change.

Error Handling Expectations

The coding agent should implement clear error handling.

Scripts and services should fail with useful messages when:

- Required configuration is missing.
- A required file does not exist.
- A required port is unavailable.
- A service is not reachable.
- A model endpoint fails.
- Credentials are required but missing.
- A deployment profile is invalid.
- A validation check fails.

Errors should identify the failing component and the next file or command to inspect.

Security Expectations

The coding agent should avoid unsafe defaults.

It should not:

- Commit secrets.
- Print secrets in logs.
- Print secrets in commit messages
- Mount broad host paths without reason.
- Grant unnecessary container privileges.
- Expose services publicly by default.
- Disable security controls without explanation.
- Delete persistent data during normal deployment.
- Add destructive commands without explicit flags.

Destructive operations should require explicit user action.

Reporting Requirements

At the end of a coding-agent task, the report should include:

- Files changed.
- Services affected.
- Configuration changed.
- Validation performed.
- Validation result.
- Documentation updated.
- Known limitations.
- Next recommended step.

The report should distinguish between completed work and deferred work.

Stop Conditions

The coding agent should stop and report when:

- Required information is missing.
- A validation step fails and the cause is not clear.
- A requested change conflicts with the architecture.
- A change would require broad rewiring beyond the current phase.

- A destructive action would be required.
- A secret or credential is needed.
- A new dependency would significantly change the deployment model.

The coding agent should not hide unresolved failures and continue building additional layers on top of a broken phase.

Summary

The coding agent should treat aMMare as a phased, configuration-driven, service-chain architecture.

It should build incrementally, validate each layer, preserve rewiring flexibility, keep scripts modular, avoid hardcoding, and update documentation as part of every implementation phase. This approach allows the project to grow from a minimal local deployment into a more complete multi-model routing engine without losing maintainability.

Memory, Context, and Retrieval Strategy

The aMMare platform may support memory, context management, and retrieval as optional capabilities.

These capabilities should be introduced only after the core model path is stable. The initial system should first prove that LangChain, the local model endpoint, LiteLLM, cloud model providers, and routing logic work correctly. Memory and retrieval should not be added early if they make troubleshooting the base service chain more difficult.

Strategy Principle

Memory and retrieval should improve response quality without making model behavior unpredictable or difficult to debug.

The system should distinguish between:

- Session context.
- Retrieved document context.
- Persistent memory.
- Runtime task state.

- Tool execution results.
- User-provided files or project content.

These should not all be treated as the same type of context.

Session Context

Session context is the short-term conversation or workflow state used during an active interaction.

It may include:

- User request.
- Recent conversation turns.
- Current task plan.
- Tool results.
- Validation results.
- Files inspected during the current workflow.
- Current routing decision.
- Current safety profile.

Session context should be temporary unless explicitly stored elsewhere.

Runtime Task State

Runtime task state tracks the current workflow inside the middleware.

It may include:

- Active phase or step.
- Pending tool requests.
- Approved actions.
- Rejected actions.
- Files modified.
- Commands executed.
- Validation status.
- Errors encountered.
- Escalation decisions.

This state is operational data. It helps the middleware manage the workflow, but it does not automatically become persistent memory.

Retrieved Context

Retrieved context is information pulled from an indexed source to support a specific request.

Sources may include:

- Project documentation.
- Architecture files.
- Repository files.
- Component documentation.
- Prior implementation notes.
- Validation output.
- Troubleshooting guides.
- User-provided documents.

Retrieved context should be relevant, limited, and traceable to its source.

The system should avoid injecting large amounts of unrelated context into prompts.

Persistent Memory

Persistent memory is information intentionally saved for future use.

It should be treated carefully because stale or incorrect memory can degrade future model behavior.

Persistent memory may include:

- Long-term project preferences.
- Stable architecture decisions.
- Reusable environment assumptions.
- Confirmed naming conventions.
- Accepted deployment defaults.
- Known constraints.
- Repeated user preferences.

Persistent memory should not automatically store every conversation, log, file, or tool result.

Memory Storage Rules

The platform should define what may be stored and what should not be stored.

Memory should not store:

- Raw secrets.
- API keys.
- Passwords.
- Private keys.
- Sensitive tokens.
- Temporary errors that are no longer relevant.
- Large unfiltered logs.
- Unverified assumptions.
- Data that belongs only to a single task.
- Generated content that was rejected or superseded.

Memory should prefer stable, confirmed information.

Retrieval Backend

If retrieval is enabled, the platform may use a vector database or another indexed retrieval backend.

The retrieval backend should be optional.

Possible stored data may include:

- Documentation chunks.
- Architecture notes.
- Component guides.
- Repository summaries.
- Troubleshooting notes.
- Known issue records.
- Validation history summaries.

The retrieval backend should support rebuild or reindex operations so stale content can be corrected.

Embedding Model

Retrieval may require an embedding model.

The embedding model may be local or remote depending on deployment policy.

For local-first deployments, a local embedding model is preferred when practical. This reduces external dependency and keeps project content local.

Embedding configuration should be externalized in the same way as model routing configuration.

Example configuration areas:

```
config/models.yaml  
config/retrieval.yaml  
env/ammare-memory.env
```

Context Injection Policy

Retrieved context should be injected into prompts according to policy.

The policy should define:

- Which sources are eligible for retrieval.
- How many chunks may be injected.
- Maximum context size.
- Source priority.
- Whether retrieved context must be cited internally.
- Whether stale content should be excluded.
- Whether sensitive content must be redacted.
- Whether retrieval can be disabled per task.

The system should favor smaller, higher-quality context over large context dumps.

Source Tracking

Retrieved context should be traceable.

When the model uses retrieved context, the middleware should track:

- Source file.
- Chunk identifier.
- Retrieval score if available.
- Timestamp or index version.

- Whether the source is user-provided, generated, or system documentation.

This helps debug poor responses and makes retrieval behavior easier to improve.

Memory and Routing Interaction

Memory and retrieval may influence model routing, but they should not bypass routing policy.

For example:

- A simple request with strong retrieved context may stay on a local model.
- A complex request with many retrieved sources may escalate to a stronger model.
- Sensitive retrieved context may force local-only routing.
- Large context requirements may require a model with a larger context window.

Routing policy should remain explicit and configurable.

Memory and Tool Safety

Memory should not bypass the tool execution safety model.

Stored context may help the model understand the environment, but it should not grant permission to execute tools, access files, call APIs, or modify infrastructure.

Tool execution should still follow:

- Approved tool registry.
- Default deny rules.
- Human approval gates.
- File access controls.
- Secret handling.
- Audit logging.

Memory improves context. It does not expand authority.

Indexing Process

The indexing process should be controlled and repeatable.

Recommended script examples:

```
scripts/index-docs.sh
scripts/index-repo.sh
scripts/rebuild-retrieval-index.sh
scripts/validate-retrieval.sh
```

Indexing scripts should:

- Load retrieval configuration.
- Identify eligible files.
- Exclude secrets and ignored paths.
- Chunk content consistently.
- Generate embeddings.
- Store index metadata.
- Report indexed sources.
- Report skipped sources.

The coding agent should not silently index everything in the repository without clear rules.

Retrieval Validation

Retrieval should have its own validation process.

Validation should confirm:

- Retrieval backend is running if enabled.
- The embedded model is reachable.
- Documents can be indexed.
- Queries return relevant results.
- Disabled retrieval does not break the core service chain.
- Sensitive files are excluded.
- Reindexing works.

Retrieval quality should be tested separately from model quality.

Reset and Purge Behavior

Memory and retrieval data should be removable.

The platform should provide a safe way to:

- Clear session state.
- Rebuild retrieval indexes.
- Remove indexed documents.
- Purge persistent memory.
- Reset memory services.

Destructive reset operations should require explicit flags.

Example:

```
scripts/reset-memory.sh --purge
```

Normal uninstall should not delete memory or retrieval data unless explicitly requested.

Documentation Requirement

Memory and retrieval behavior should be documented.

Documentation should explain:

- What is stored.
- What is not stored.
- Where data is stored.
- How retrieval is configured.
- How documents are indexed.
- How memory is cleared.
- How retrieval is disabled.
- How to troubleshoot bad retrieval results.
- How memory affects routing and prompts.

This is especially important because memory-related behavior can be difficult to understand if it is not documented clearly.

Summary

Memory, context, and retrieval should be optional, controlled, and introduced only after the core aMMare service chain is stable.

The system should clearly separate session context, runtime task state, retrieved context, and persistent memory. Retrieval should be source-tracked, configurable, and validated independently. Memory should improve continuity without storing secrets, bypassing safety controls, or making model behavior difficult to troubleshoot.

Deployment Model

The aMMare platform should be deployed as a containerized, local-first service stack.

The initial target environment is a local lab or development host. The platform should not require public inbound access, external orchestration, or a cloud deployment to function. Cloud model providers may be added later as optional backends, but the core system should remain usable with local services.

Deployment Goals

The deployment model should support:

- Local-first operation.
- Containerized services.
- Modular component deployment.
- Static local IPs or local DNS.
- Optional cloud model providers.
- Optional memory and retrieval services.
- Repeatable deployment through scripts.
- Independent validation of each service.
- Simple troubleshooting and rollback.

The deployment should begin small and expand only as each phase is validated.

Initial Deployment Target

The initial deployment should assume a single local host running the required containers.

The minimum deployment includes:

```
ammare-langchain
ammare-local-llm
```

This creates the first working request path:

```
User/API Client → LangChain → Local LLM
```

This model is intentionally simple. It validates the core middleware-to-model path before additional routing or gateway components are introduced.

Expanded Deployment Target

Later deployments may add:

```
ammare-litellm
ammare-headroom
ammare-openhands
ammare-vector-db
ammare-embedding
```

The expanded deployment may support:

```
User/API Client → LangChain → Headroom → LiteLLM → Local LLM / Cloud LLM
```

Memory, retrieval, and OpenHands should remain optional and should be enabled only when the core model-routing path is stable.

Container Runtime

The platform should run each major component as a separate containerized service.

The deployment may use Podman, Docker , or another compatible container runtime depending on the target host. The selected runtime should be documented in the

implementation repository. Docker has been selected for the POC as the target system is Ubuntu.

Each service should define:

- Container name.
- Image source.
- Published ports.
- Environment files.
- Config mounts.
- Data mounts.
- Log behavior.
- Health check.
- Restart behavior.

The coding agent should avoid assuming that all services run inside one container.

Network Model

The default network model should be local and private.

Supported patterns include:

- Single-host container networking.
- Static local lab IP addresses.
- Local DNS names.
- Container service names.
- Host-published local ports.
- Multi-host local lab deployment if needed.

Public inbound exposure should not be required.

If cloud model providers are enabled, the deployment may require outbound internet access from the host or routing layer.

IP Address and DNS Strategy

The platform may use either local IP addresses or local DNS names.

For the local lab deployment, static IP addresses are acceptable and may be simpler to operate.

However, service targets should still be assigned through configuration variables such as:

```
AMMARE_MODEL_GATEWAY_URL=
```

This allows the backend path to change without modifying application code.

Persistent Data

Persistent data should be stored in predictable project or host paths.

Examples:

```
data/models/  
data/vector-db/  
data/openhands-workspaces/  
logs/
```

Persistent data should not be deleted during normal redeployment or uninstall unless the user explicitly requests a destructive purge.

Configuration and Secrets

Deployment-specific values should be stored in environment files and structured config files.

Secrets should be stored separately from committed example files.

The deployment should clearly distinguish between:

- Example environment files.
- Local environment files.
- Structured config files.
- Secret files.
- Runtime-generated files.

No raw secrets should be committed to the repository. No secrets in commit messages.

Deployment Profiles

The deployment model should support profiles.

Examples:

```
minimal-local  
local-with-litellm  
hybrid-local-cloud  
full-agentic  
full-agentic-with-memory
```

Each profile should define which services are enabled and how the service chain is connected.

The default profile should be the simplest working deployment.

Host Assumptions

The initial host should provide:

- Supported Linux operating system.
- Container runtime.
- Sufficient CPU and memory.
- GPU support if required by the local model.
- Local storage for model files.
- Local network access between enabled services.
- Optional outbound internet access for cloud providers.

Exact hardware requirements should be documented after the initial model runtime is selected.

Deployment Boundary

The initial deployment should not attempt to solve every production concern.

Deferred or optional areas may include:

- Kubernetes deployment.
- High availability.
- Multi-node orchestration.
- Enterprise identity integration.
- Centralized observability stack.
- Production ingress design.
- Multi-user authorization model.

These may be considered later if the project matures beyond the local lab deployment.

Summary

The aMMare deployment model is local-first, containerized, modular, and profile-driven.

The first deployment should prove the smallest useful service chain. Later profiles can add LiteLLM, cloud providers, Headroom, OpenHands, memory, and retrieval. Static local IPs are acceptable for the lab environment, but service relationships should remain configurable so the deployment can evolve without code rewrites.

Validation and Success Criteria

The aMMare platform should define clear validation steps and success criteria for each development phase.

A phase should not be considered complete simply because files were created or containers started. Each phase must prove that the intended behavior works, that failures are understandable, and that documentation has been updated.

Primary Success Criteria

The primary success criteria for aMMare is to prove that the platform can function as a practical agentic coding assistant endpoint.

The end user should be able to configure an IDE, coding assistant, or agentic CLI tool to use the aMMare gateway as its model or agent endpoint. After connecting to that endpoint, the user should be able to submit prompts and receive clear, human-readable responses.

Beyond basic text generation, the platform should also validate controlled agentic behavior. When authorized by policy and user approval, the system should be able to perform practical development and troubleshooting tasks such as:

- Inspecting project files.
- Creating new files.
- Modifying existing files.
- Generating patches or diffs.
- Running approved scripts.

- Running tests or validation commands.
- Inspecting command output.
- Reviewing logs.
- Tailing logs for troubleshooting.
- Checking service status.
- Identifying likely failure points.
- Recommending or applying corrective changes.
- Summarizing what was changed and what validation was performed.

The platform should not only answer questions about code or infrastructure. It should be able to participate in a controlled development workflow where the model reasons about the task, requests approved tool actions, receives tool results, and continues troubleshooting or implementation based on observed system state.

A successful validation result should prove that aMMare can operate as more than a model proxy. It should behave as a controlled agentic middleware layer capable of supporting real coding, deployment, and troubleshooting workflows while preserving safety controls, auditability, and user approval for sensitive actions.

Validation Principles

Validation should follow these principles:

- Validate one layer before adding the next.
- Test components independently.
- Test the active service chain end to end.
- Use repeatable scripts instead of manual commands where possible.
- Return clear pass/fail output.
- Preserve logs for troubleshooting.
- Avoid validating optional components when they are disabled.
- Confirm documentation matches the implemented behavior.
- Stop before adding complexity if the current phase does not pass.

The project should treat failed validation as a phase blocker.

Phase-Level Success Criteria

Each phase should define its own success criteria.

At minimum, a phase is successful when:

- Required files are present.
- Required configuration is documented.
- Required services start successfully.
- Required ports are reachable.
- Health checks pass.
- Component validation scripts pass.
- Active service-chain validation passes.
- Logs show expected behavior.
- Errors are handled clearly.
- Documentation has been updated.
- No secrets are committed.

If a phase introduces a new component, that component should have deployment, validation, logging, and troubleshooting support.

Configuration Validation

Configuration validation should confirm that the active deployment profile is usable before services are deployed.

Validation should check:

- Required environment files exist.
- Required variables are set.
- Enabled components have required configuration.
- Disabled components are not required by the active profile.
- Ports are valid and available.
- Paths exist or can be created.
- Model registry entries are valid.
- Routing rules reference known models.
- Secrets are present when cloud providers are enabled.
- No known secret files are committed.

Recommended script:

```
scripts/validate-config.sh
```

Component Validation

Each major component should have a validation script.

Examples:

```
scripts/validate-local-llm.sh
scripts/validate-langchain.sh
scripts/validate-litellm.sh
scripts/validate-headroom.sh
scripts/validate-openhands.sh
scripts/validate-memory.sh
```

Component validation should confirm:

- Container or service is running.
- Required port is listening.
- Health endpoint responds if available.
- Required config files are mounted or readable.
- Required upstream service is reachable.
- Logs do not show fatal startup errors.
- A minimal functional test succeeds where practical.

Service-Chain Validation

The active service chain should be validated end to end.

Recommended script:

```
scripts/validate-service-chain.sh
```

This script should read the active deployment profile and validate the correct path.

Examples:

minimal-local:

```
LangChain → Local LLM
```

local-with-litellm:

```
LangChain → LiteLLM → Local LLM
```

hybrid-local-cloud:

```
LangChain → LiteLLM → Local LLM / Cloud LLM
```

full-agentic:

```
LangChain → Headroom → LiteLLM → Local LLM / Cloud LLM
```

The validation result should identify which layer failed if the full chain does not pass.

Model Routing Validation

When routing is enabled, routing behavior should be validated separately from basic service health.

Validation should confirm:

- Default model route works.
- Local model route works.
- Cloud model route works if enabled.
- Disabled providers are not selected.
- Escalation rules select the expected target.
- Fallback rules behave as expected.
- Routing decisions are logged.
- Routing configuration references valid models.

Recommended scripts:

```
scripts/validate-routing.sh
scripts/show-routing.sh
scripts/test-routing.sh
```

Tool Execution Validation

If tool execution is enabled, validation should confirm that tool access is controlled.

Validation should check:

- Approved tools are available.
- Unapproved tools are denied.
- File access is scoped correctly.
- Write actions require approval when configured.
- Dangerous commands are blocked.
- Secrets are not returned in tool output.
- Tool execution is logged.
- Failed tools return clear errors.

Tool validation should be conservative. It should prove that safety controls work, not only that tools can execute.

Memory and Retrieval Validation

If memory or retrieval is enabled, it should be validated independently.

Validation should confirm:

- Memory service is running if required.
- Vector database is reachable if enabled.
- Embedding model is reachable if enabled.
- Documents can be indexed.
- Retrieval returns relevant results.
- Sensitive files are excluded.
- Retrieval can be disabled without breaking the core service chain.
- Memory or index data can be reset when explicitly requested.

Recommended scripts:

```
scripts/validate-memory.sh  
scripts/validate-retrieval.sh  
scripts/rebuild-retrieval-index.sh
```

Deployment Validation

A successful deployment should produce a clear summary.

The summary should include:

- Active deployment profile.
- Enabled services.
- Skipped services.
- Active model gateway URL.
- Published ports.
- Validation results.
- Log locations.
- Next recommended command.

A deployment should not be considered successful if the script completes but the active service chain fails validation.

Documentation Validation

Documentation should be checked as part of phase completion.

Documentation should accurately describe:

- Current service names.
- Current ports.
- Current environment variables.
- Current deployment profiles.
- Current service-chain paths.
- Current validation scripts.
- Current troubleshooting process.
- Current limitations.

If implementation and documentation disagree, the phase should be considered incomplete.

Final System Success Criteria

The final system should be considered successful when:

- The selected deployment profile deploys cleanly.
- All enabled services start successfully.
- All required health checks pass.
- The active service chain passes end-to-end validation.
- Local model routing works.
- Cloud model routing works if enabled.
- Escalation and fallback behavior match policy.
- Optional components can be disabled cleanly.
- Logs are useful for troubleshooting.
- Helper scripts work as documented.
- Documentation is complete enough for a new user or coding agent to continue safely.
- No secrets are committed.
- Destructive actions require explicit user intent.

Failure Criteria

A phase should not proceed if:

- Required services do not start.
- Health checks fail.
- The active service chain fails.
- Configuration is unclear or duplicated.
- Required scripts are missing.
- Logs do not identify the failure point.
- Documentation is stale.
- Secrets are exposed.
- The implementation requires manual undocumented steps.

Failures should be fixed or documented before moving to the next phase.

Summary

Validation is a required part of the aMMare development model.

Each phase should produce a working, documented, and testable system state. Component validation proves individual services work. Service-chain validation proves the active request

path works. Final success requires the deployment, routing, helper scripts, documentation, and safety controls to behave consistently with the architecture.

Deferred v2 Features

The initial aMMare implementation should focus on proving the core local-first, multi-model routing architecture.

Some features are valuable but should be deferred until the base platform is stable, documented, and validated. Deferring these items keeps the first implementation achievable and reduces troubleshooting complexity.

Deferral Principle

A feature should be deferred when it adds complexity without being required to prove the core architecture.

Deferred features may be revisited after the platform can reliably:

- Deploy the minimal local service chain.
- Route through LiteLLM.
- Add at least one cloud model provider.
- Apply routing and escalation rules.
- Validate the active service chain.
- Use modular deployment scripts.
- Provide clear documentation and troubleshooting guidance.

Deferred Features List

The following features should be considered v2 or later unless explicitly pulled into the initial scope.

Feature	Reason for Deferral
Kubernetes deployment	Adds orchestration complexity before the local container model is proven.
High availability	Not required for the initial local lab deployment.

Multi-node production deployment	Requires additional networking, service discovery, storage, and security planning.
Enterprise identity integration	Valuable later, but not required for the first working system.
Multi-user authorization model	The initial platform can assume a single trusted operator.
Web UI	Useful, but API/script-driven operation should be validated first.
Full observability stack	Metrics and logging are useful, but a dedicated observability deployment can come later.
Advanced memory automation	Memory should remain optional until the core routing path is stable.
Advanced agent collaboration	Multi-agent coordination should wait until single-agent workflows are reliable.
Automated production actions	The initial system should not directly modify production infrastructure.
Complex policy engine	Basic configuration-driven policy should be proven first.
Distributed task queue	Not needed until workflows become long-running or concurrent.
Advanced RBAC	Requires a clearer multi-user operating model.
External plugin marketplace	Too broad for the initial architecture.
Automated self-update mechanism	Should wait until deployment and rollback behavior are mature.

Not Deferred

The following capabilities are not deferred because they are required for the initial architecture:

- Local LLM endpoint.
- LangChain middleware.
- Logical model gateway configuration.
- LiteLLM routing layer.
- At least one local model route.

- At least one optional cloud model route.
- Basic routing and escalation configuration.
- Modular deployment scripts.
- Component validation scripts.
- Service-chain validation script.
- Component documentation.
- Configuration documentation.
- Troubleshooting documentation.
- Secret exclusion from committed files.

These are part of the minimum architecture required to prove aMMare.

Conditional Features

Some features may be optional in the initial implementation but should be planned for structurally.

Examples:

- Headroom.
- OpenHands.
- Memory and retrieval.
- Vector database.
- Embedding model.
- Cloud model providers beyond the first.
- Optional UI.

These features should not block the minimal local deployment, but the architecture should avoid choices that make them difficult to add later.

Re-Evaluation Criteria

Deferred features may be reconsidered when:

- The core service chain is stable.
- Deployment scripts are repeatable.
- Validation scripts are reliable.
- Documentation matches implementation.
- Routing and escalation behavior is working.
- Troubleshooting paths are clear.
- The added feature solves a real operational need.

- The added feature can be tested independently.

A deferred feature should not be added only because it is interesting. It should have a clear purpose and a validation plan.

Summary

Deferred v2 features are intentionally excluded from the first implementation so the project can focus on a reliable, local-first multi-model routing engine.

The initial release should prove the core chain, configuration model, routing behavior, modular deployment, validation scripts, and documentation. More advanced platform features can be added after the foundation is stable.

Client Tool Integration Strategy

The aMMare platform should be easy to use from common IDEs, editors, and agentic coding tools.

A primary goal of the project is to reduce the barrier to entry for end users who want to route their coding assistant workflows through the aMMare gateway. Users should not need to manually discover endpoint formats, configuration paths, model names, or API compatibility settings for each tool.

The platform should provide both documentation and helper scripts for configuring supported tools.

Integration Goal

The end user should be able to configure a supported client tool to use aMMare as its model gateway or API endpoint.

Supported tools may include:

- Zed.
- Continue plugin for VS Code.
- Native VSCode chat or compatible model-provider configuration.
- Cline.
- OpenHands.
- Aider.

- Other OpenAI-compatible clients (GUI/CLI) that allow bring-your-own (BYO) Endpoints

The exact level of support may vary by tool, but the project should aim to provide a clear path for each supported integration.

Gateway Compatibility

Where practical, the aMMare gateway should expose an OpenAI-compatible API surface.

This improves compatibility with existing tools that already support OpenAI-compatible endpoints, custom base URLs, or local model providers.

The documentation should clearly define:

- Gateway base URL.
- Required API path.
- Model name or alias to use.
- Authentication requirements, if any.
- Supported request formats.
- Known limitations by tool.
- Whether streaming is supported.
- Whether tool/function calling is supported.
- Whether file operations are handled by the client, aMMare middleware, or both.

The goal is not to make every client behave identically. The goal is to make the expected behavior clear and easy to configure.

Client Configuration Scripts

The project should include helper scripts that configure supported tools to use aMMare where possible.

Recommended script layout:

```
scripts/integrations/  
  configure-zed.sh  
  configure-continue-vscode.sh  
  configure-vscode-chat.sh  
  configure-cline.sh  
  configure-openhands.sh  
  configure-aider.sh
```

These scripts should:

- Read the active aMMare gateway endpoint from configuration.
- Detect whether the target tool is installed where practical.
- Locate the expected user configuration path.
- Back up existing client configuration before modifying it.
- Add or update the aMMare provider entry.
- Avoid overwriting unrelated user settings.
- Print the final endpoint and model alias being configured.
- Clearly report any manual steps still required.

If a tool's configuration format changes or cannot be safely modified, the script should stop and provide manual instructions instead of making unsafe assumptions.

Manual Configuration Documentation

Each supported client tool should have a short integration document.

Recommended layout:

```
docs/integrations/  
  zed.md  
  continue-vscode.md  
  vscode-chat.md  
  cline.md  
  openhands.md  
  aider.md
```

Each document should include:

- What the tool is used for in the aMMare workflow.
- Whether the integration is fully supported, partially supported, or experimental.
- Required aMMare deployment profile.
- Required gateway URL.
- Required model alias.
- Required authentication setting.
- Manual configuration steps.
- Validation prompt.

- Troubleshooting notes.
- Link to the upstream tool documentation.

The project may link to external installation and product documentation instead of recreating full upstream documentation. The aMMare documentation should focus on what is specific to connecting that tool to the aMMare gateway.

Example Configuration Values

The documentation should provide consistent example values.

Example local gateway:

```
AMMARE_GATEWAY_URL=http://10.1.10.17:8080
```

Example OpenAI-compatible base URL:

```
http://10.1.10.17:8080/v1
```

Example model alias:

```
ammare-default
```

Example local-only model alias:

```
ammare-local
```

Example high-reasoning model alias:

```
ammare-high-reasoning
```

The actual values should be defined in the active configuration files and deployment profile. Examples should not become hardcoded assumptions.

Integration Profiles

The project may define integration profiles for common client use cases.

Examples:

Profile	Intended Use
basic-chat	Simple prompt and response through the aMMare gateway.
coding-assistant	IDE-based code explanation, editing, and review.
agentic-coding	Tool-assisted coding workflows with file inspection and command execution.
local-only	Uses only local model backends.
hybrid-routing	Allows routing between local and cloud model backends.

These profiles should map to documented model aliases and gateway behavior.

Tool-Specific Notes

Different client tools may interact with aMMare differently.

For example:

- Some tools may only need an OpenAI-compatible base URL and model name.
- Some tools may manage file edits locally and only use aMMare for model responses.
- Some tools may support agentic workflows but require separate permission settings.
- Some tools may support custom providers but not full tool calling.
- Some tools may require manual configuration because their settings are not safely scriptable.

The integration documentation should identify these differences clearly.

Validation for Client Integrations

Each client integration should include a validation process.

Validation should confirm:

- The client can connect to the aMMare gateway.
- The configured model alias is accepted.
- A basic prompt returns a readable response.
- Streaming works if supported.
- Coding prompts work as expected.
- Agentic tool behavior works where supported.
- Errors are clear when the gateway is unavailable.
- The client is using aMMare and not silently falling back to another provider.

Recommended validation prompt:

Confirm that you are responding through the aMMare gateway. Then summarize the active model route if that information is available.

For coding tools, validation may also include a simple repository-local task such as inspecting a README, explaining a script, or generating a small non-destructive file.

Safety and Permissions

Client integration should not bypass the aMMare safety model.

Even if a client tool supports autonomous file edits or command execution, aMMare should still enforce its own tool execution policy, approval gates, logging, and scope restrictions.

The documentation should explain where permissions are controlled:

- In the client tool.
- In the aMMare gateway.
- In the LangChain middleware.
- In the local workspace.
- In the operating system or container runtime.

Users should understand whether a task is being executed by the client directly or by aMMare-controlled tooling.

Documentation Requirement

The main README should include a short “Supported Client Tools” section.

That section should link to the detailed integration documents and identify the fastest path for a new user.

Example:

```
For the lowest-friction setup, deploy the minimal or hybrid profile, then  
run the matching integration script for your preferred coding tool.
```

The README should not contain every configuration detail. It should point users to the correct integration document.

Summary

The aMMare client integration strategy is intended to make the platform easy to adopt from existing coding tools.

The project should provide scripts and documentation for tools such as Zed, Continue for VS Code, native VS Code chat, Cline, OpenHands, and Aider. The goal is to let users quickly configure their preferred IDE or agentic CLI to use aMMare as the gateway endpoint while preserving clear documentation, validation steps, and safety boundaries.